



DATABASE DESIGN HEALER DEALER PHARMACY

By

DATA DYNAMOS

REINE BAZ

reine.baz@lau.edu

MAY EZZEDDINE

may.ezzeddine@lau.edu

REIN KHALIFEH

rein.khalifeh01@lau.edu

SIMYAA JABER

simiyaa.jaber@lau.edu

GEORGY ZAOUK

georgy.zaouk@lau.edu

A REPORT

submitted to Dr. Ramzi R. Haraty in partial fulfillment of the requirements
for the course “CSC375: Database Management Systems” in Computer Science

Phase II

November 15, 2024

Table of Contents

<i>Table of Contents.....</i>	<i>2</i>
<i>Introduction.....</i>	<i>6</i>
<i>Copyright Notice</i>	<i>7</i>
<i>System Description and Constraints.....</i>	<i>8</i>
<i>ER Diagram for Healer Dealer Pharmacy</i>	<i>9</i>
<i>Entities.....</i>	<i>10</i>
1. EMPLOYEE.....	10
2. DEPENDENT	11
3. BRANCH	12
4. INVENTORY	13
5. SUPPLIER.....	14
6. SELF_CARE_HYGIENE_PRODUCT.....	15
7. MEDICINE.....	16
8. PRESCRIPTION	16
9. CUSTOMER	18
10. BILLING_INVOICE.....	19
<i>Relationships.....</i>	<i>20</i>
1. has1.....	20
2. works_in	20
3. manages.....	21
4. contains1	22
5. stores	22
6. supplies	23
7. includes.....	24
8. contains2	25
9. has2.....	25
10. bills	26
11. issues	27
<i>ER to Relational Mapping Algorithms.....</i>	<i>28</i>
STEP 1: Mapping of Regular Entity Types	28
1. EMPLOYEE.....	28
2. BRANCH	29

3.	INVENTORY	29
4.	SUPPLIER.....	29
5.	SELF_CARE_HYGIENE_PRODUCT	30
6.	MEDICINE.....	30
7.	PRESCRIPTION	30
8.	BILLING_INVOICE.....	30
Step 2: Mapping of Weak Entity types		31
1.	CUSTOMER	31
2.	DEPENDENT.....	31
Step 3: Mapping of Binary 1:1 Relationship Types		32
1.	BRANCH (<i>manages</i>).....	32
2.	INVENTORY (<i>contains</i>).....	32
STEP 4: Mapping of Binary 1:N Relationship Types		33
1.	DEPENDENT (<i>has1</i>)	33
2.	PRESCRIPTION (<i>has2</i>).....	33
3.	BILLING_INVOICE (<i>bills, issues</i>).....	33
4.	EMPLOYEE (<i>works_in</i>).....	34
Step 5: Mapping of M:N Relationship Types		34
1.	contains2.....	35
Step 6: Mapping Multivalued Attributes.....		35
1.	Payment_method	35
2.	Item_ID.....	35
3.	Dosage_instructions	36
4.	Payment_term.....	36
Step 7: Mapping of N-ary Relationship Types		36
1.	Supplies	37
2.	Includes.....	37
3.	Stores	37
FINAL STEP: Final Displays		38
1.	EMPLOYEE.....	38
2.	BRANCH	38
3.	INVENTORY	38
4.	SUPPLIER.....	38
5.	SELF_CARE_HYGIENE_PRODUCT	39
6.	MEDICINE.....	39
7.	PRESCRIPTION	39
8.	BILLING_INVOICE.....	39
9.	CUSTOMER	39
10.	DEPENDENT	39
11.	contains2	39
12.	Payment_method.....	39
13.	Item_ID	39
14.	Dosage_instructions	40
15.	Payment_term	40
16.	Supplies.....	40
17.	Includes	40
18.	Stores.....	40

Table Structure for Healer Dealer Pharmacy..... 41

CREATE TABLES:	41
1- BRANCH:.....	41
2- EMPLOYEE:	42
3- SUPPLIER:.....	42
4- BILLING_INVOICE:	43
5- INVENTORY:.....	43
6- CUSTOMER:.....	43
7- MEDICINE:.....	44
8- SELF_CARE_HYGIENE_PRODUCT:	44
9- PRESCRIPTION:	44
10- DEPENDENT:	45
11- Payment_term:	45
12- contains2:	45
13- Payment_method:	45
14- Item_ID:.....	46
15- Dosage_Instructions:.....	46
16- Supplies:.....	46
17- Includes:.....	46
18- Stores:	47
ALTER command:	47

Table Descriptions:	47
1-BRANCH:.....	47
2- EMPLOYEE:	48
3-SUPPLIER:.....	48
4- BILLING_INVOICE:	49
5- INVENTORY:.....	49
6- CUSTOMER.....	50
7- MEDICINE:.....	50
8- SELF_CARE_HYGIENE_PRODUCT:	50
9- PRESCRIPTION:	51
10- DEPENDENT	51
11- PAYMENT_TERM.....	52
12- CONTAINS2.....	52
13- PAYMENT_METHOD	52
14- ITEM_ID	53
15- DOSAGE_INSTRUCTIONS	53
16- SUPPLIES	53
17- INCLUDES.....	53
18- STORES;.....	54

TABLE STATE..... 54

Inserting Values:	54
1. BRANCH:	54
2. EMPLOYEE:	55
3. INVENTORY:	56
4. SUPPLIER:	56
5. CUSTOMER:.....	57
6. BILLING_INVOICE:	58
7. DEPENDENT:	60

8. Payment_term:	60
9. MEDICINE:	61
10. SELF_CARE_HYGIENE_PRODUCT:.....	61
11. PRESCRIPTION:	62
12. Contains2:	63
13. Payment_method:.....	63
14. Item_ID:	64
15. Dosage_Instructions:	65
16. Supplies:	65
17. Includes:	66
18. Stores:.....	66
TABLE STATES:	67
1. EMPLOYEE	67
2. BRANCH	67
3. SUPPLIER	68
4. INVENTORY	68
5. CUSTOMER	68
6. BILLING_INVOICE	69
7. DEPENDENT	69
8. PAYMENT_TERM	69
9. MEDICINE	70
10. SELF_CARE_HYGIENE_PRODUCT.....	70
11. PRESCRIPTION	71
12. CONTAINS2	71
13. PAYMENT_METHOD	72
14. ITEM_ID	72
15- DOSAGE_INSTRUCTIONS	73
16- SUPPLIES	73
17- INCLUDES.....	74
18- STORES.....	74
Transactions	76
Query 1: Identifying and Removing Expired Medicines	76
Problem:	76
Query:.....	76
Query 2: A new customer purchases items	79
Problem:	79
Query:.....	79
Query 3: A new customer comes with a prescription	80
Problem:	81
Query:.....	81
Query 4: List All Employees and Their Assigned Branches	82
Query:.....	83
Query 5: Calculate Total Sales for Each Medicine	84
Problem:	84
Query:.....	84
Query 6: Add a New Branch and Hiring a Manager.....	85
Problem:	85

Query:.....	86
Query 7: Dosage Instructions for a Prescription	87
Problem:	87
Query:.....	87
QUERY 8: Medication Recall Management	88
Problem:	88
Query:.....	88
QUERY 9: Supplier Payment Summary for Auditing	89
Problem:	89
Query:.....	90
QUERY 10: High-Risk Customer Detection	92
Problem:	92
Query.....	95
<i>Normalization Up to The BCNF Normal Form:</i>	<i>96</i>
1. EMPLOYEE.....	99
2. BRANCH	99
3. SUPPLIER.....	100
4. BILLING_INVOICE	101
5. CUSTOMER	101
6. MEDICINE.....	102
7. PRESCRIPTION	102
8. INVENTORY	104
9. SELF_CARE_HYGIENE_PRODUCT.....	104
10. DEPENDENT	105
11. STORES	106
12. SUPPLIES	106
<i>Conclusion.....</i>	<i>107</i>

Introduction

We, The Data Dynamos, take pleasure in submitting our report titled “A Database Design for Healer Dealer Pharmacy”. This project has enhanced our learning experience and allowed us to understand databases in depth.

In our report, a summary of our work process is provided. We were assigned to create a database for a pharmacy, and we created “Healer Dealer Pharmacy”. Through this experience, we got to know both the biology and the computer science of our field of study from a practical side. We have gained experience through using Lucidchart to design an ER model. This report will explore different aspects that are necessary in the process of creating “Healer Dealer Pharmacy” and its different branches.

Copyright Notice

Copyright © 2024 The Data Dynamos. This work is protected by Copyright, and permission should be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. To obtain permission(s) to use material from this work, please submit a written request to one of the authors of this work: rein.khalifeh01@lau.edu, reine.baz@lau.edu, simiyaa.jaber@lau.edu, may.ezzeddine@lau.edu, georgy.zaouk@lau.edu .

System Description and Constraints

The Healer Dealer Pharmacy is a busy, effective facility that provides a large selection of medications, health supplies, and individualized pharmacy services to maintain community health. Focusing on dependability and accessibility, Healer Dealer Pharmacy makes sure that clients receive the appropriate goods, assistance, and direction for their health. Quality care and seamless experience for each visitor are the top priorities for the committed pharmacists and customer service professionals that work at the pharmacy.

Healer Dealer Pharmacy's advanced inventory management system is the foundation of its business operations. Every medication and self-care item that is kept in stock is tracked by this system, which closely monitors supplier details, expiration dates, and quantity. Healer Dealer Pharmacy ensures that shelves are consistently stocked with new, safe products that are available to satisfy the needs of its clients by using automatic stock-level updates and timely alerts for expired items.

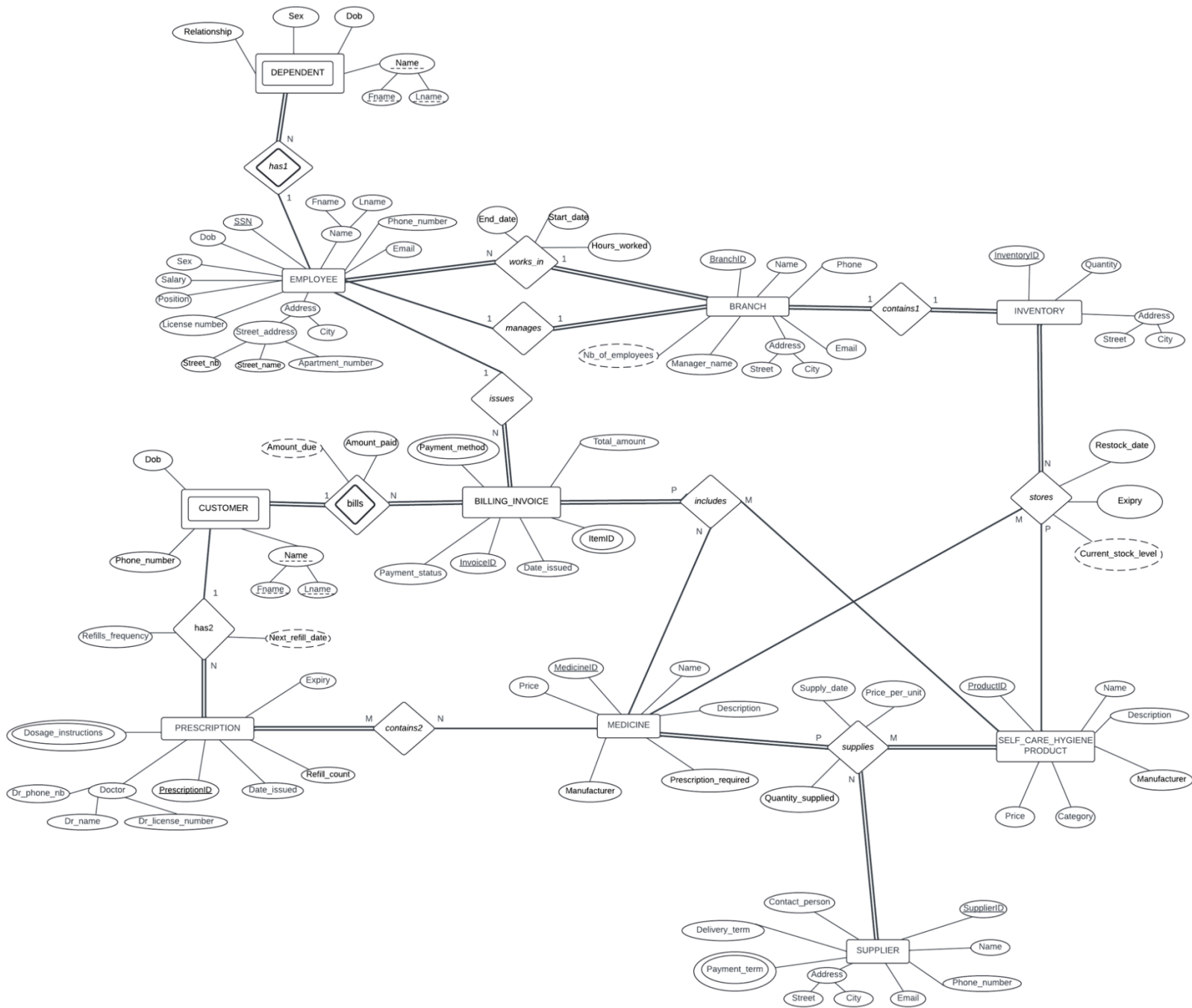
The prescription management tool is another crucial component of the system. With precise information on dose, refill requirements, and any required verifications, this component keeps track of every prescription. The system verifies the prescription information for controlled or prescription-only drugs to make sure that clients are only given the drugs they are permitted to consume. This meticulous procedure keeps consumer safety at the top of the priority list and helps prevent misuse.

Additionally, the pharmacy has a thorough billing and invoicing system that makes it simple to keep track of payment methods, statuses, and balances. Customers who depend on Healer Dealer Pharmacy for their healthcare requirements will benefit from this feature's assurance that all transactions are accurate and transparent, which fosters trust and convenience.

However, running such a complicated organization has its own set of difficulties. Due to inventory restrictions, every item—especially those with limited shelf lives—must be closely watched, necessitating an accurate and responsive system. Another top objective is data security and privacy; the system must adhere to stringent confidentiality guidelines to safeguard sensitive data, including personal information and prescription medication information. Lastly, maintaining correct and current financial records is essential to the pharmacy's seamless and dependable operation. This is achieved by prompt payment tracking.

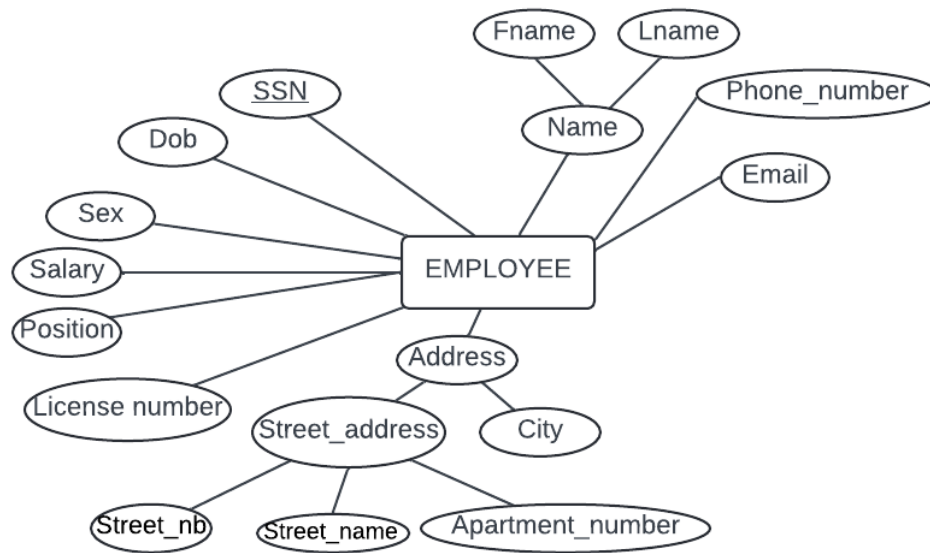
Healer Dealer Pharmacy is a lifeline for the community, blending expertise, care, and technology to create a safe, welcoming, and efficient space for all who walk through its doors.

ER Diagram for Healer Dealer Pharmacy



Entities

1. EMPLOYEE



Employee is the employed staff of the Healer Dealer Pharmacy. They include the pharmacists, the cleaning staff, the cashiers, the accountants, and the like... The primary key of this entity type **EMPLOYEE** is SSN which is the social security number. This is a unique nine-digit number for every person, and thus for every employee. Other attributes are used to describe the **EMPLOYEE** entity type:

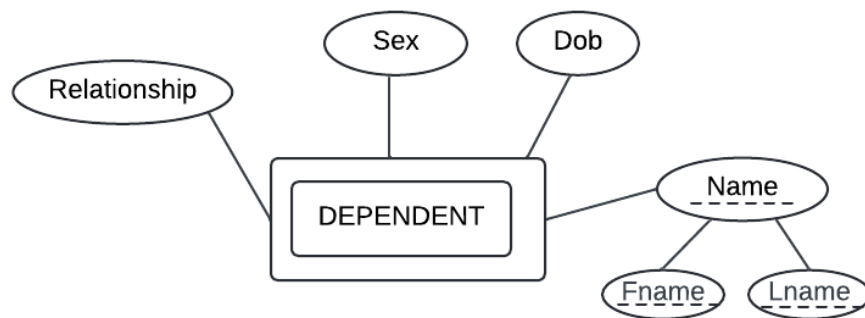
Simple attributes include:

- **Dob:** This is the date of birth of the employee, formatted strictly as such: DD/MM/YYYY.
- **Sex:** Stores the gender of the employee (male, female).
- **Salary:** Includes the salary of the employee.
- **Position:** There are different positions for every employee, such as cleaner, cashier, accountant, pharmacist, intern, and manager.
- **Phone_number:** This is the personal number of the employee. Its format should be: +961 XX XXX XXX
- **Email:** Stores the employee's work email which must end with @healerdealer.com
- **License_number:** This is the license number unique to the pharmacist position. Any other position will have a NULL value in this entry.

Composite attributes include:

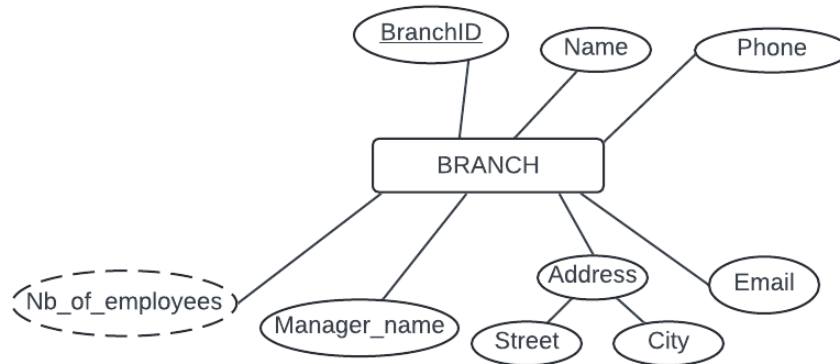
- **Name:** The employee's name. It is a composite attribute that includes 2 sub-attributes **Fname**, for the first name, and **Lname**, for the last name.
- **Address:** Stores the address of the employee, and it includes the sub-attributes **Street_address** and **City** that they are residing in. **Street_address** is also composite with 3 sub-attributes: **Street_nb** (of the street), **Street_name** (name of the street), and **Apartment_number**.

2. DEPENDENT



DEPENDENT is a weak entity. It is the employee's dependents or basically all those who depend on the employee financially. It is thus identified by entity type **EMPLOYEE** and a partial key **Name**, a composite attribute with sub-attributes **Fname** and **Lname** for the first name and last name, respectively. It also includes simple attributes **Sex** (describing the dependent's sex), **Dob** (date of birth), and **Relationship** (their relationship with the employee as in father, mother, sister, wife,...).

3. BRANCH



Healer Dealer Pharmacy has several branches across Lebanon. The branch is uniquely identified by the **BranchID**. And so, we include this as our primary key for the **BRANCH** entity type in our pharmacy database. The following attributes provide further information about the branch:

Simple attributes include:

- ***Name***: This is the branch name.
- ***Phone***: This is the number to call the branch.
- ***Email***: This is the email address to contact the branch.
- ***Manager_name***: This is the name of the branch's manager.

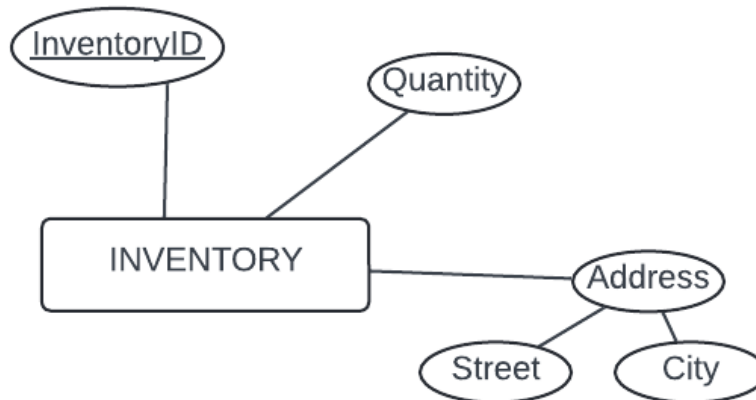
Composite attributes include:

- ***Address***: This composite attribute includes 2 sub-attributes: ***Street*** and ***City***. This is the address of the branch.

Derived attributes include:

- ***Nb_of_employees***: This is the number of employees that work at the branch, it is derived by counting the number of employees from **EMPLOYEE** entity type that have a relationship "***works_in***" with each **BranchID**.

4. INVENTORY



The inventory is the pharmacy's storage of supplies/products. It is uniquely identified by the **InventoryID**. And so we include this as our primary key for the **INVENTORY** entity type in our pharmacy database. The other attributes that provide information about the branch are:

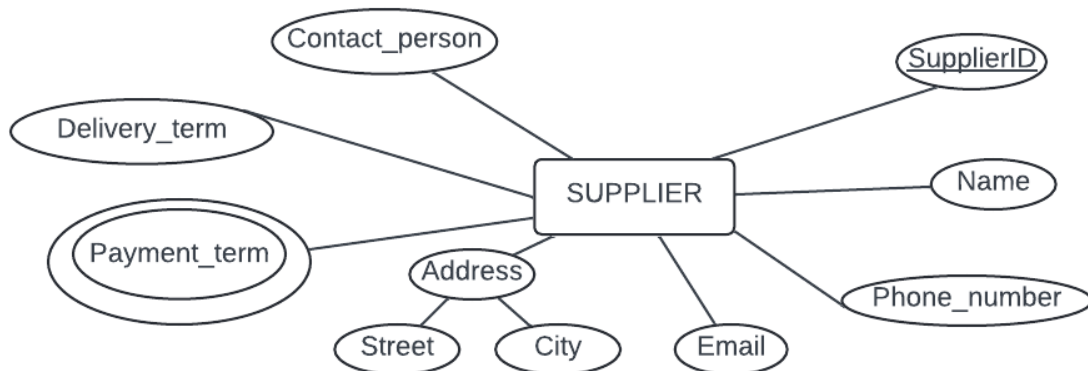
Simple attributes include:

- ***Quantity***: Stores the quantity of each item in an inventory.

Composite attributes include:

- ***Address***: This composite attribute includes 2 sub-attributes: ***Street*** and ***City***. This is the address of the inventory.

5. SUPPLIER



The supplier is the company that is responsible for supplying all our pharmacy branches with products, such as medicine and self-care or hygiene products. The supplier company is uniquely identified by the **SupplierID** at the local tax office or commercial registry. And so we include this as our primary key for the **SUPPLIER** entity type in our pharmacy database. The other attributes that provide information about the suppliers are:

Simple attributes include:

- **Delivery_term**: How often are the products delivered as in weekly, monthly, or yearly.
- **Name**: This is the supplier company name.
- **Contact_person**: This stores the name of the person we contact in that supplier company.
- **Phone_number**: This is the phone number of the contact person.
- **Email**: The supplier company's email. It must be formatted as @domainname.com

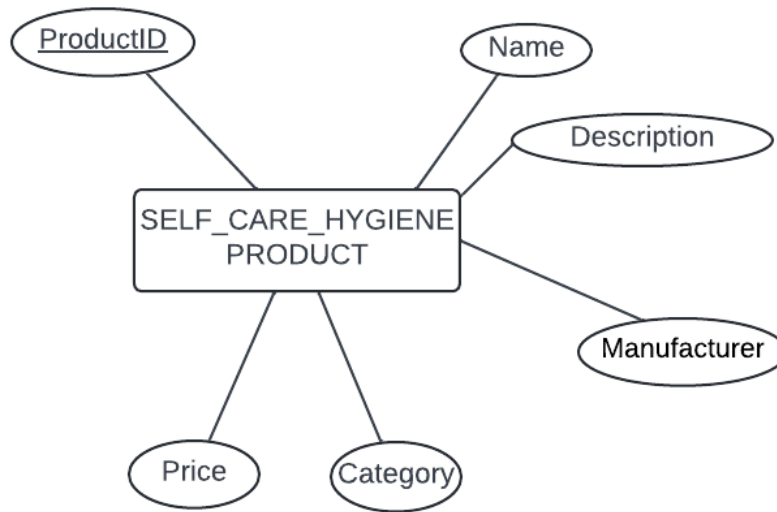
Composite attributes include:

- **Address**: This is a composite attribute that includes 2 sub-attributes: **Street** and **City**. This is the address of the supplier company.

Multivalued attributes include:

- **Payment_term**: The payment terms of the supplier which can include: cash in advance, letter of credit, documentary collection or draft, open account, and consignment. This is a multivalued attribute that can contain multiple payment terms per supplier, and they are written as a string value with breaks represented by a "\n".

6. SELF_CARE_HYGIENE_PRODUCT

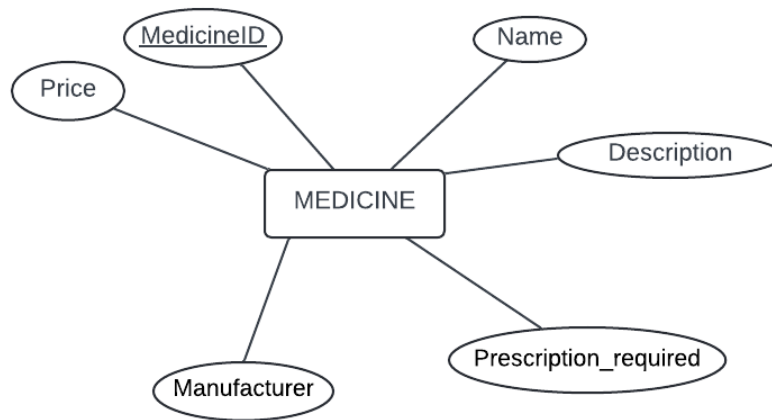


The **SELF_CARE_HYGIENE_PRODUCT** entity comprises products designed for personal care and general health maintenance. Unlike medications, these products are typically over-the-counter (OTC) items that don't require prescriptions, covering a range of hygiene, skincare, and personal wellness products. Attributes include:

Simple attributes:

- **ProductID**: A unique identifier for each product, so it will be the primary key for this entity type.
- **Name**: Text field identifying the specific product name.
- **Price**: A numeric value indicating the cost per unit, in dollars.
- **Description**: A brief explanation of the product's use.
- **Category**: Text or selection field for classifying the product, such as dental, baby, makeup...
- **Manufacturer**: The product's manufacturer company name.

7. MEDICINE

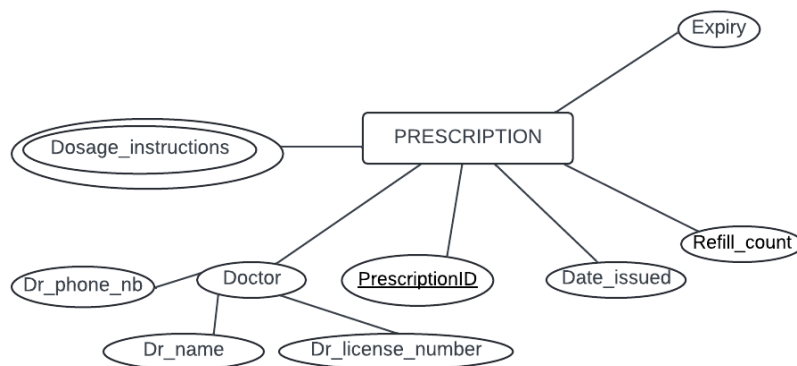


The **MEDICINE** entity represents pharmaceutical products designed to diagnose, treat, prevent, or alleviate health conditions. Attributes within this entity help define each medicine's unique characteristics, such as dosage, manufacturer, and prescription requirements, ensuring each product's correct classification and use. Attributes include:

Simple attributes:

- **MedicineID**: A unique identifier for each medicine. This will be the primary key.
- **Name**: Stores the name of the medicine.
- **Description**: Text field providing details about the medicine.
- **Price**: A numeric value indicating cost per unit, in dollars.
- **Prescription_required**: Boolean indicating if a prescription is needed.
- **Manufacturer**: This stores the medicine's manufacturer company name.

8. PRESCRIPTION



The **PRESCRIPTION** entity documents orders issued by healthcare professionals authorizing specific medications or treatment regimens for patients. This entity ensures that medications requiring medical authorization are tracked, managed, and dispensed following regulations. Attributes include:

Simple attributes:

- **PrescriptionID**: Unique identifier for each prescription, thus will be used as a primary key to identify each prescription in this entity type.
- ***Date_issued***: Date field indicating when the prescription was created.
- ***Expiry***: The date when the prescription expires.
- ***Refill_count***: Integer indicating the total number of allowed refills.

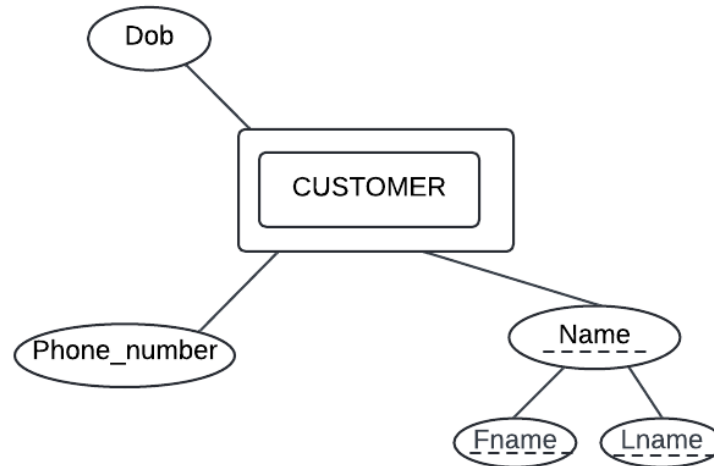
Composite attribute:

- ***Doctor***: This composite attribute describes the prescriber doctor with the following sub-attributes: ***Dr_name***, ***Dr_license_number***, and ***Dr_phone_nb***.

Multivalued attributes:

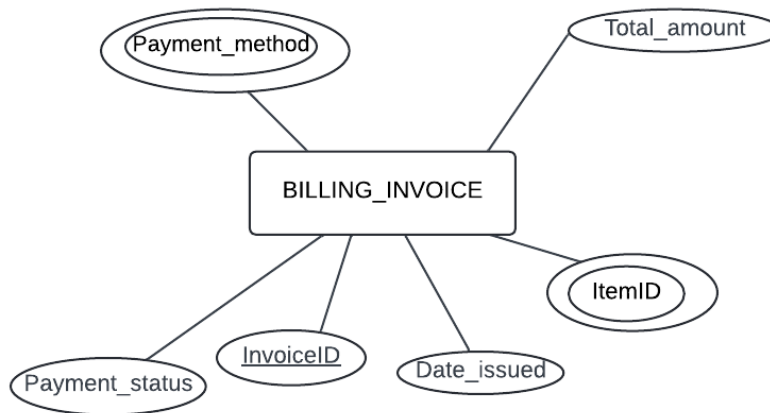
- ***Dosage_instructions***: Contains detailed instructions on multiple different medicines stored in it.

9. CUSTOMER



CUSTOMER is another weak entity. It represents the customers that register or purchase products from our pharmacy. It is a weak entity type in this database since the Healer Dealer Pharmacy won't take extensive information on the customer. The partial key is the **Name** of the customer, which is a composite attribute with sub-attributes **Fname** and **Lname** for the first name and last name, respectively. It will also be identified by the BILLING_INVOICE key entity type. It includes simple attributes: **Dob** (date of birth) and **Phone_number**.

10. BILLING_INVOICE



After a customer purchases a product from the pharmacy, a billing invoice is issued to them. The billing invoice is uniquely identified by the **InvoiceID**. And so we include this as our primary key for the **BILLING_INVOICE** entity type in our pharmacy database. The other attributes that provide information about the branch are:

Simple attributes:

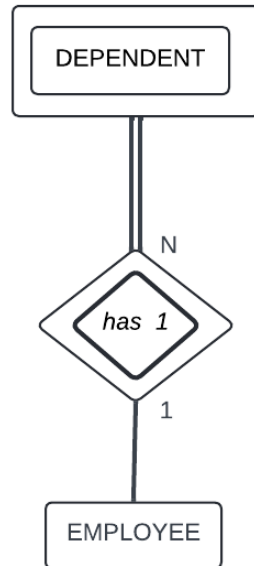
- **Total_amount:** This is the total amount to be paid in dollars, as LBP is an unstable currency.
- **Payment_status:** Records whether the customer paid the bill completely or not. A paid bill stores the status “complete”, and an unpaid one has the status “pending”.
- **Date_issued:** This is the date the billing invoice was issued.

Multivalued attributes:

- **Payment_method:** The method by which the customer is paying. There are different payment methods: card, cash, voucher...
- **ItemID:** This is a unique identifier for each item bought by the customer. A billing invoice can have multiple items.

Relationships

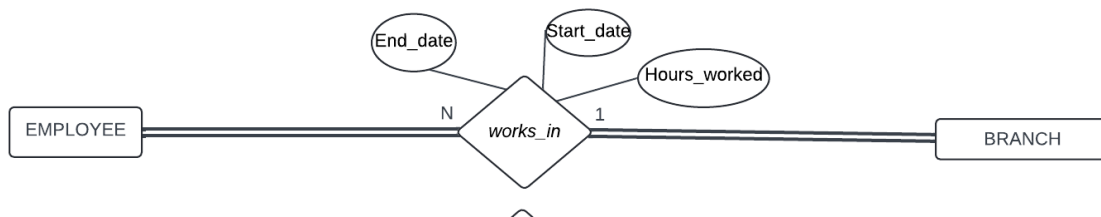
1. *has1*



An employee may have several dependents. So a “*has1*” identifying relationship was created between the **EMPLOYEE** entity and the **DEPENDENT** weak entity, with the relationship identifying this weak entity. The participation is partial on the **EMPLOYEE** side and total on the **DEPENDENT** side since all dependents belong to an employee, but not all employees have dependents.

Regarding the cardinality ratio, it is a one-to-many (1:N) for **EMPLOYEE** to **DEPENDENT** since an **EMPLOYEE** “*has1*” one or more **DEPENDENT**, but a **DEPENDENT** “*has1*” one **EMPLOYEE** only.

2. *works_in*



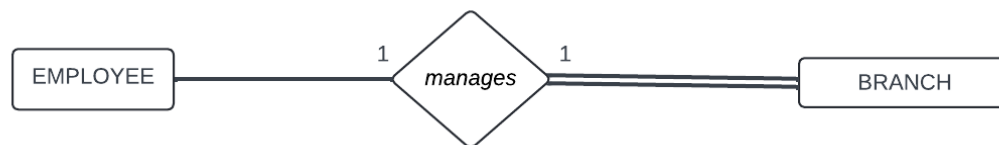
All employees will work in different branches of the pharmacy, so a relationship “*works_in*” was created between **EMPLOYEE** and **BRANCH**. The participation is total on both sides as all employees will work in a branch and all branches will have employees.

The cardinality ratios are many to one (N:1) for **EMPLOYEE** to **BRANCH** since an **EMPLOYEE** “*works_in*” one **BRANCH** only, but one **BRANCH** can have many **EMPLOYEE**.

The attributes of the relationship “*works_in*” include simple attributes:

- **Start_date**: This is the first day of employment of the employee at a specific branch, written in date format DD/MM/YYYY
- **End_date**: This is the last day of employment or date of transfer, written in date format DD/MM/YYYY. If the employee has not left yet, then the value may be NULL.
- **Hours_worked**: Shows the shift timings of the employee.

3. *manages*



An employee manages each branch of the Healer Dealer Pharmacy around Beirut. So, a “*manages*” relationship must be created between the two entities: **BRANCH** and **EMPLOYEE**. The participation is total on the **BRANCH** side but partial on the **EMPLOYEE** side since every branch is managed by an employee, but not all employees can manage a branch.

The cardinality ratio of relation is one-to-one (1: 1) since only one **EMPLOYEE** “*manages*” a **BRANCH**, and every **BRANCH** is managed by an **EMPLOYEE**.

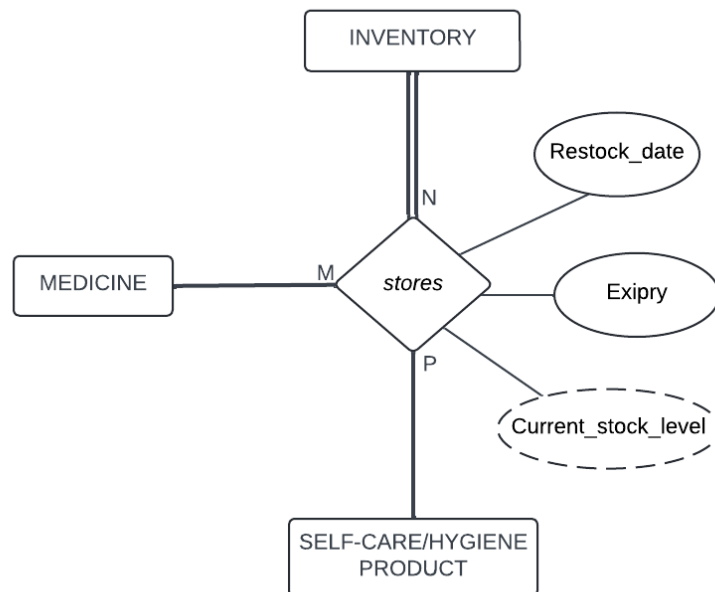
4. *contains1*



Each branch of the pharmacy contains a unique inventory, storing its stock of medicinal and hygiene products. Thus, a “*contains1*” relationship should be created between the two strong entities, **BRANCH** and **INVENTORY**. Participation is total on both sides, as every branch must have an inventory, and each inventory is exclusively assigned to a branch.

The cardinality ratio of the relationship is one-to-one (1:1) since a **BRANCH** “*contains1*” an **INVENTORY** and only one **INVENTORY** is linked to a **BRANCH**.

5. *stores*



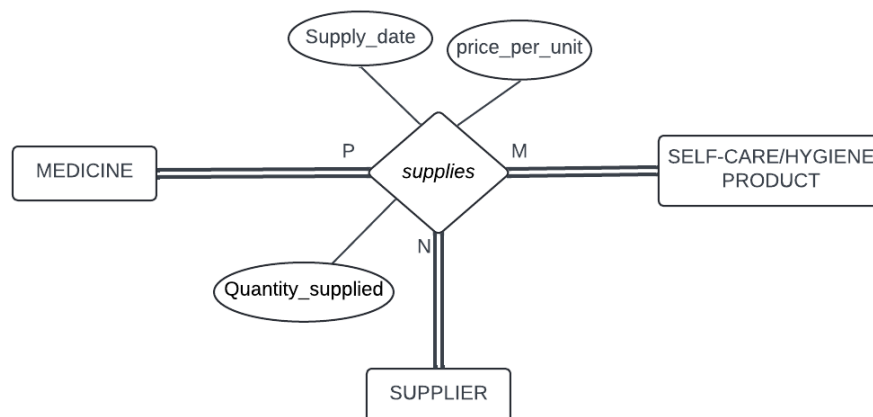
The Healer Dealer Pharmacy has multiple inventories that store medicine and self-care/hygiene products. So, a ternary relationship “*stores*” will connect the three entities: **INVENTORY**, **MEDICINE**, and **SELF_CARE_HYGIENE_PRODUCT**. **INVENTORY** has total participation in the relationship, while **MEDICINE** and **SELF_CARE_HYGIENE_PRODUCT** will have partial participation as all **INVENTORY** entities “*stores*” at least one **MEDICINE** and **SELF_CARE_HYGIENE_PRODUCT**, but not all **MEDICINE** and **SELF_CARE_HYGIENE_PRODUCT** will be available in **INVENTORY**.

The cardinality ratio is many to many to many (N:M:P) for **INVENTORY** to **MEDICINE** to **SELF_CARE_HYGIENE_PRODUCT** since an **INVENTORY** “stores” one or many **MEDICINE** and **SELF_CARE_HYGIENE_PRODUCT**, and a type of medicine or a product can be stored in multiple inventories.

The following attributes are used to provide more information on this relationship:

- **Restock_date**: Date when the stock was last replenished.
- **Expiry**: Tracks the expiration date of all medicine and product batches.
- **Current_stock_level**: A derived attribute that tracks the current number of units of medicine or products in stock by subtracting from itself the units sold (stored in the *itemID* attribute in **BILLING_INVOICE** entity type) and adding to itself the newly supplied stock (stored in *quantity_supplied* attribute of the relationship “supplies”).

6. supplies



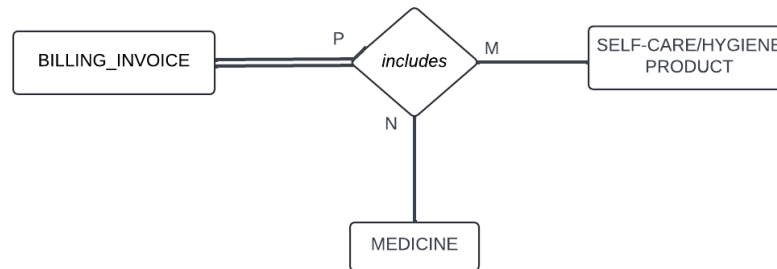
All suppliers will supply medicine and/or self-care/hygiene products. As a result, the ternary relationship “supplies” was created between the three entity types **SUPPLIER**, **MEDICINE**, and **SELF_CARE_HYGIENE_PRODUCT**. All 3 sides have total participation as all suppliers supply medicine and products and all products and medicine are supplied by the suppliers.

The cardinality ratio is many to many to many since a **SUPPLIER** “supplies” one or many **MEDICINE** and/or one or many **SELF_CARE_HYGIENE_PRODUCT**, and a **MEDICINE** or **SELF_CARE_HYGIENE_PRODUCT** can be supplied by many **SUPPLIER**.

This relationship includes the following attributes:

- ***Supply_date***: Stores the date the supplier last provided a product.
- ***Price_per_unit***: The price at which the supplier sells each unit to the pharmacy.
- ***Quantity_supplied***: Keeps track of the number of units supplied in each transaction.

7. *includes*



A billing invoice is given when medicine and/or self-care/hygiene products are purchased. Thus, a ternary *"includes"* relation should be done between the three entity types: **BILLING_INVOICE**, **MEDICINE**, and **SELF_CARE_HYGIENE_PRODUCT**. The participation is total on the **BILLING_INVOICE** side while partial on both **MEDICINE** and **SELF_CARE_HYGIENE_PRODUCT** sides; this is because all billing invoices will include medicine and/or self-care/hygiene products and not all products will be included in the billing invoice.

The ratio is many-to-many-to-many (N: M: P) since a **BILLING_INVOICE** *"includes"* one or many **MEDICINE** and/or one or many **SELF_CARE_HYGIENE_PRODUCT**, and a product, whether from **MEDICINE** or **SELF_CARE_HYGIENE_PRODUCT**, can be included in multiple **BILLING_INVOICE**.

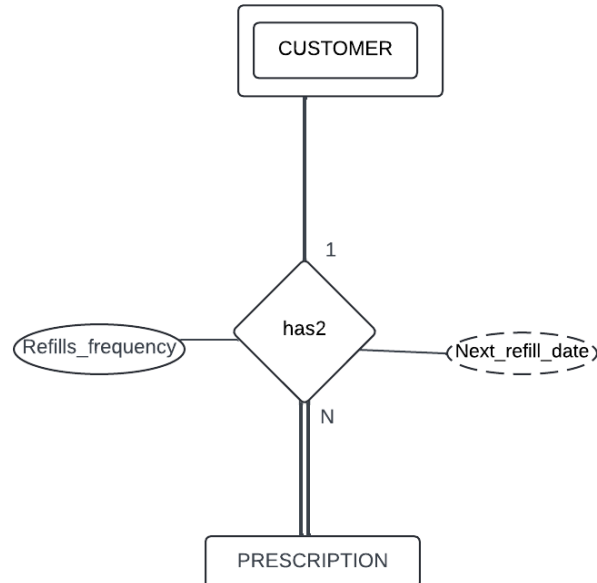
8. contains2



Each prescription issued by the pharmacy contains one or more medicines required for a patient's treatment. Thus, a *"contains2"* relationship should be created between the two entities, **PRESCRIPTION** and **MEDICINE**. Participation is total on the **PRESCRIPTION** side, as each prescription must contain at least one medicine. However, participation is partial on the **MEDICINE** side, as medicine may exist in the pharmacy without being prescribed.

The cardinality ratio of the relationship is many-to-many (N: M), as a **PRESCRIPTION** *"contains2"* multiple **MEDICINE**, and each **MEDICINE** can be included in multiple **PRESCRIPTIONS**.

9. has2



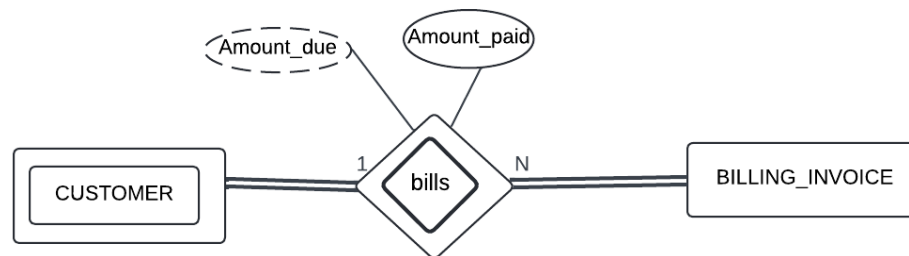
A customer will have one or more prescriptions. So, a relationship *"has 2"* was created between the entities **PRESCRIPTION** and **CUSTOMER**. The participation is partial on the customer side while total on the prescription side since all prescriptions must be provided by a customer, but not all customers have a prescription.

The cardinality ratio is one-to-many (1:N) for **CUSTOMER** to **PRESCRIPTION** since a **CUSTOMER** “has2” one or many **PRESCRIPTION**, but a **PRESCRIPTION** “has2” only one **CUSTOMER**.

The relationship “has2” has two attributes:

- **Refills_frequency**: how often the prescribed medicine must be refilled. (e.g., every 10 days)
- **Next_refill_date**: This derived attribute tracks the customer’s next date to refill their medication. It is derived from the **refill_frequency** attribute and the **PRESCRIPTION** entity’s attributes **date_issued** and **refill_count**. From the **date_issued**, the medication must be refilled every time after the period shown in **refill_frequency**, until the number of refills reaches the refill count. The refill date is calculated according to the above rules.

10. bills



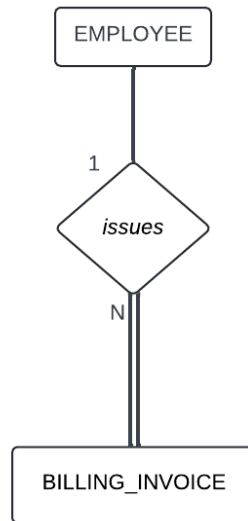
Customers will receive a billing invoice after purchasing Healer Dealer Pharmacy products, so an identifying relationship “bills” was created between weak entity type **CUSTOMER** and strong entity **BILLING_INVOICE** with total participation on the **CUSTOMER** side and total participation on the **BILLING_INVOICE** side since all billing invoices will bill a customer, and all customers will buy products and thus all will be billed.

The cardinality ratio is one to many (1:N) since a **CUSTOMER** can receive multiple **BILLING_INVOICE** but a **BILLING_INVOICE** “bills” only 1 **CUSTOMER**.

The attributes include the following:

- **Amount_paid**: This documents the amount the customer paid in the billing invoice.
- **Amount_due**: This is a derived attribute that tracks how much of the payment is left. It is derived by subtracting **amount_paid** from **total_amount**.

11. issues



An employee issues a billing invoice to the customer. So, a relationship “*issues*” was created between the entity types **EMPLOYEE** and **BILLING_INVOICE**, in which **EMPLOYEE** has partial participation while **BILLING_INVOICE** has total participation since all invoices are issued by an employee but not all employees can issue invoices, like the cleaner.

The ratio is one to many (1:N) since one **EMPLOYEE** “*issues*” multiple **BILLING_INVOICE**, but a **BILLING_INVOICE** must be issued by one **EMPLOYEE** only.

ER to Relational Mapping Algorithms

We have established an ER schema of our Healer Dealer Pharmacy database, representing it in a high-level and conceptual model comprised of entities, attributes, and relationships. However, now we must take it a step further and translate it into a more logical design, and that is the relational database design. This ER to relational design mapping is achieved through the following seven-step algorithm:

STEP 1: Mapping of Regular Entity Types

For the first step, we must create a relation or table for each regular (strong) entity type. This relation will contain all the simple attributes of its corresponding entity type and any composite attribute will be represented as a set of simple attributes. A key attribute of the entity type will also be chosen as the primary key of its relation and will be underlined.

The regular entity types in this database design for Healer Dealer Pharmacy are:

- **EMPLOYEES**
- **BRANCH**
- **INVENTORY**
- **SUPPLIER**
- **SELF_CARE_HYGIENE_PRODUCT**
- **MEDICINE**
- **PRESCRIPTION**
- **BILLING_INVOICE**

1. EMPLOYEE

<u>SSN</u>	Dob	Sex	Salary	Position
License_number	Street_nb	Street_name	Apartment_number	City
Email	Phone_number	Lname	Fname	

EMPLOYEE is a strong entity type containing both simple and composite attributes. It will be mapped as the relation seen above, which includes all simple attributes and takes the key attribute of the **EMPLOYEE** entity type, SSN, as its primary key that is underlined. The entity

type has 2 composite attributes, *Name* and *Address*. The relation will include only the simple attributes of both:

- Composite attribute *Name* will be included in the relation as the 2 simple attributes *Fname* and *Lname*.
- Composite attribute *Address* will be included in the relation as the 4 simple attributes *Street_nb*, *Street_name*, *Apartment_number*, and *City*.

2. BRANCH

<u>BranchID</u>	Name	Phone	Email	Manager_name
Street	City			

The **BRANCH** entity contains simple, composite, and derived attributes. The derived attribute *Nb_of_employees* is not represented in this relation. This relation only includes all simple attributes and the primary key *BranchID* which is underlined. The **BRANCH** entity has *Address* as a composite attribute of which only the simple attributes *Street* and *City* are included in the relation.

3. INVENTORY

<u>InventoryID</u>	Quantity	Street	City
--------------------	----------	--------	------

The **INVENTORY** entity contains simple and composite attributes. This relation only includes all simple attributes and the primary key *InventoryID* which is underlined. The **INVENTORY** entity has *Address* as a composite attribute of which only the simple attributes *Street* and *City* are included in the relation.

4. SUPPLIER

<u>SupplierID</u>	Name	Contact_person	Delivery_term	Email
Phone_number	Street	City		

The **SUPPLIER** entity type contains simple, composite, and multivalued attributes. So, we create a relation containing all its simple attributes and underline this relation's primary key *SupplierID*, which is the key attribute of the **SUPPLIER** entity type. In this relation, the entity's composite attribute *Address* is

included as 2 simple attributes: *Street* and *City*. The multivalued attribute *Payment_term* is not included in this relation but will be considered in the algorithm's sixth step.

5. SELF_CARE_HYGIENE_PRODUCT

<u>ProductID</u>	Name	Description	Category	Price
Manufacturer				

The **SELF_CARE_HYGIENE_PRODUCT** entity contains simple attributes. This relation only includes all simple attributes and the primary key *ProductID*, which is underlined.

6. MEDICINE

<u>MedicineID</u>	Name	Description	Prescription_required	Price
Manufacturer				

The **MEDICINE** entity contains simple attributes, so we create a relation that includes all these simple attributes and has *MedicineID* as the primary key, which is underlined.

7. PRESCRIPTION

<u>PrescriptionID</u>	Date_issued	Refill_count	Expiry	Dr_phone_nb
Dr_name	Dr_license_number			

The **PRESCRIPTION** entity contains simple, composite and multivalued attributes. The multivalued attribute *Dosage_instructions* is not represented in this relation. This relation only includes all simple attributes and the primary key *PrescriptionID* which is underlined. The **PRESCRIPTION** entity has *Doctor* as a composite attribute of which only the simple attributes *Dr_phone_nb*, *Dr_name* and *Dr_license_number* are included in the relation.

8. BILLING_INVOICE

<u>InvoiceID</u>	Date_issued	Total_amount	Payment_status
------------------	-------------	--------------	----------------

The entity type **BILLING_INVOICE** contains both simple and multivalued attributes. The multivalued attributes *Payment_method* and *ItemID* are not included in this relation. This

relation only includes all the simple attributes, with the entity's key attribute **InvoiceID** underlined and taken as the relation's primary key.

Step 2: Mapping of Weak Entity types

The second step entails mapping the weak entity types into relations. Each weak entity type will have a relation that will include only all its simple attributes, while composite attributes are included as simple attributes. In addition, the relation will also contain a foreign key attribute referencing the primary key of its identifying strong entity type. The primary key of the relation will be the combination of the foreign key and the partial key of the weak entity type. Currently, the Healer Dealer database contains 2 weak entity types:

1. CUSTOMER

<u>Fname</u>	<u>Lname</u>	<u>Bill_ID</u>	Phone_number	Dob
--------------	--------------	----------------	--------------	-----

The **CUSTOMER** weak entity contains simple and composite attributes. Its relation will include all simple attributes, and its composite attribute **Name** will be included as two simple attributes: **Fname** and **Lname**. The relation will also contain a foreign key called **Bill_ID**, referencing the primary key of its identifying entity type **BILLING_INVOICE**. The primary key of this relation is a combination of the foreign key and the partial keys **Fname** and **Lname**, which are underlined.

2. DEPENDENT

<u>Fname</u>	<u>Lname</u>	<u>ESSN</u>	Sex	Dob
Relationship				

The **DEPENDENT** weak entity contains simple and composite attributes. Its relation will include all simple attributes, and its composite attribute **Name** will be included as two simple attributes: **Fname** and **Lname**. The relation will also contain a foreign key called **ESSN**, the primary key of its identifying entity type **EMPLOYEE**. The primary key of this relation is a combination of the foreign key and the partial keys **Fname** and **Lname**, which are underlined.

Step 3: Mapping of Binary 1:1 Relationship Types

After mapping the entity types into the relational model, we must map the relationships. In this step, we will map the binary one-to-one relationships by following 1 of the 3 approaches. The first method, known as the foreign key approach, is used when there is a relationship with at least one entity having total participation. In this case, we add a foreign key to the relation of the entity type that has total participation, which references the primary key of the entity type with partial participation. The second approach is called the merged relation approach, and it is used only when both entity types of the relationship have total participation. And so, a relation is created by merging the two entity types and the relationship. The third approach, called the cross-reference or relationship relation approach, is where we create a third relation which will include the primary keys of both entities participating in the relationship. In this case, we relied on the foreign key approach for both 1:1 relationships in our database:

1. BRANCH (*manages*)

<u>BranchID</u>	Name	Phone	Email	Street
City	Manager_name	Manager_SSN		

Every branch has a manager, thus we established a “*manages*” relationship between **EMPLOYEE** and **BRANCH**. Using the foreign key approach, we map this relationship by adding a foreign key, *Manager_SSN*, to the relation of the entity type with total participation: **BRANCH**. This foreign key is renamed from the primary key of **EMPLOYEE**, *SSN*.

2. INVENTORY (*contains*)

<u>InventoryID</u>	Quantity	Street	City	B_ID
--------------------	----------	--------	------	------

Every branch contains an inventory. Thus, we established a “*contains*” relationship between **INVENTORY** and **BRANCH**. Both entity types have total participation, meaning that we could use either of the 2 approaches: the foreign key approach or the merged relation approach. We opted for the foreign key approach by adding the foreign key, *B_ID*, renamed from the primary key of **BRANCH** to the **INVENTORY** relation (since both entity types have total participation, it doesn’t matter where we add the foreign key).

STEP 4: Mapping of Binary 1:N Relationship Types

In this stage, we will model binary one-to-many relationships. This involves adding a foreign key to the entity on the “many” side of the relationship. The foreign key will reference the primary key of the entity on the “one” side. Any additional simple attributes pertinent to the one-to-many relationship will also be included. The specific one-to-many relationships to be modeled are: “*has1*”, “*has2*”, “*bills*”, “*issues*” and “*works_in*”.

1. DEPENDENT (*has1*)

<u>Fname</u>	<u>Lname</u>	<u>ESSN</u>	Sex	Dob
Relationship				

Multiple dependents can be linked to a single employee. The “*has1*” relationship connects the **DEPENDENT** and **EMPLOYEE** entities. As **DEPENDENT** is on the “many” side, we introduce a foreign key, *ESSN*, which references the primary key of the **EMPLOYEE** entity. But since we have already made that addition, then nothing has changed.

2. PRESCRIPTION (*has2*)

<u>PrescriptionID</u>	Date_issued	Refill_count	Expiry	Dr_phone_nb
Dr_name	Dr_license_number	Refills_frequency	CF_name	CL_name

Multiple prescriptions can be associated with a single customer. The “*has2*” relationship connects the **CUSTOMER** and **PRESCRIPTION** entities. As **PRESCRIPTION** is on the “many” side, we introduce a foreign key which is the primary key of the **CUSTOMER** entity and rename it: {*CF_name*, *CL_name*}. The “*has2*” relationship also has simple and derived attributes, so we add to the relation the simple attribute *Refills_frequency*. The derived attribute *Next_refill_date* is ignored.

3. BILLING_INVOICE (*bills*, *issues*)

<u>InvoiceID</u>	ESSN	Date_issued	Total_amount	Payment_status
------------------	------	-------------	--------------	----------------

CF_name	CL_name	Amount_paid
---------	---------	-------------

Multiple billing invoices can be associated with a single customer. The “*bills*” relationship connects the **BILLING_INVOICE** and **CUSTOMER** entities. As **BILLING_INVOICE** is on the “many” side, we introduce a foreign key which is the primary key of the **CUSTOMER** entity, and we rename it {*CF_name*, *CL_name*}. In addition, the “*bills*” relationship has simple and derived attributes, so we include all the simple attributes to the **BILLING_INVOICE** relation and we ignore the derived attributes for now.

A single employee can issue multiple billing invoices. The “*issues*” relationship connects the **BILLING_INVOICE** and **EMPLOYEE** entities. As **BILLING_INVOICE** is on the “many” side, we introduce a foreign key, *ESSN*, which references the primary key of the **EMPLOYEE** entity.

4. **EMPLOYEE** (*works_in*)

<u>SSN</u>	Fname	Lname	Dob	Sex
Position	License_number	Salary	Phone_number	Email
City	Street_nb	Street_name	Apartment_number	B_ID
End_date	Start_date	Hours_worked		

Multiple employees can be assigned to a single branch. The “*works_in*” relationship connects the **EMPLOYEE** and **BRANCH** entities. As **EMPLOYEE** is on the “many” side, we introduce a foreign key, *B_ID*, which references the primary key of the **BRANCH** entity. We also add to the relation all the simple attributes of the relationship.

Step 5: Mapping of M:N Relationship Types

In this step, we are going to map many-to-many relationships. For each many-to-many relationship, we need to create a new relationship that includes, as foreign keys, the primary keys of each relation. The combination of foreign keys constitutes the primary keys of the newly created relation. We must also include any other simple attribute of the many-to-many relationship. The many-to-many relationship needed to be mapped is: “*contains2*”.

1. contains2

<u>pID</u>	<u>mID</u>
------------	------------

Many prescriptions contain many medicines. The “contains2” relationship links the **PRESCRIPTION** entity and the **MEDICINE** entity. We create a new relation called “contains2” that includes the primary keys of the **PRESCRIPTION** and **MEDICINE** entities. The primary key of the **PRESCRIPTION** entity, PrescriptionID, is added to the “contains2” relation and renamed pID. Also, the primary key of the **MEDICINE** entity, MedicineID, is added to the “contains2” relation and renamed mID. The combination of both added keys represents the primary keys of the “contains2” relationship, thus being underlined.

Step 6: Mapping Multivalued Attributes

In this step, we will map the multivalued attributes that were previously ignored. We create a relation for each multivalued attribute, such that the relation will include both the related attribute and a foreign key referencing the primary key of the entity type the multivalued attribute belonged to. The primary key of this new relation will be the combination of the mentioned attributes. In the Healer Dealer database, we have 4 multivalued attributes: *Payment_method*, *Item_ID*, *Dosage_instructions*, and *Payment_term*.

1. Payment_method

<u>InvoiceID</u>	<u>Payment_method</u>
------------------	-----------------------

The multivalued attribute *Payment_method* belongs to the **INVOICE** entity type. We represent it by creating a relation that contains a foreign key *InvoiceID* that references the primary key of the **INVOICE** entity. It also includes the attribute *Payment_method* which represents the multiple methods an invoice can be paid (card, cash, voucher...). The combination of both attributes is the primary key of this relation, so both are underlined.

2. Item_ID

<u>InvoiceID</u>	<u>ItemID</u>
------------------	---------------

The multivalued attribute *ItemID* belongs to the **INVOICE** entity type as well. We represent it by creating a relation that contains a foreign key *InvoiceID* that references the primary key of the **INVOICE** entity. It also includes the attribute *ItemID* which represents the multiple items purchased and included in an invoice. The primary key of this relation is the combination of both attributes, and both are therefore underlined.

3. Dosage_instructions

<u>PrescriptionID</u>	<u>Dosage_instruction</u>
-----------------------	---------------------------

The multivalued attribute *Dosage_instruction* belongs to the **PRESCRIPTION** entity type. We represent it by creating a relation that contains a foreign key *PrescriptionID* referencing the primary key of the **PRESCRIPTION** entity. It also includes the attribute *Doasge_instruction* corresponding to the dose instructions for the many medicines recorded in the prescription. The primary key of this relation is the combination of both attributes, and both are therefore underlined.

4. Payment_term

<u>SupplierID</u>	<u>Payment_term</u>
-------------------	---------------------

The multivalued attribute *Payment_term* belongs to the **SUPPLIER** entity type. We represent it by creating a relation that contains a foreign key *SupplierID*, referencing the primary key of the **SUPPLIER** entity. It also includes the attribute *Payment_term*, which corresponds to the various payment terms that can be used for each supplier, like cash on delivery or prepaid, etc. The primary key of this relation is the combination of both attributes, and both are therefore underlined.

Step 7: Mapping of N-ary Relationship Types

In this step, we are going to map all the N-ary Relationship types. We should create a new relation containing the primary keys of all participating entities and any simple attributes of the relationship type. The N-ary relationships needed to be mapped are: “*supplies*” and “*includes*”.

1. Supplies

<u>mID</u>	<u>sID</u>	<u>prID</u>	Price_per_unit	Supply_date
Quantity_supplied				

Many suppliers supply multiple medicines and self-care/hygiene product. The “*supplies*” relation links the three entities **SUPPLIER**, **MEDICINE**, and **SELF_CARE_HYGIENE_PRODUCT**. We create a new relation called “*supplies*” in which we include the primary keys of the **SUPPLIER**, **MEDICINE**, and **SELF_CARE_HYGIENE_PRODUCT** entities. The primary key of the **SUPPLIER** entity, *SupplierID*, is added to the “*supplies*” relation renamed *sID*. Then, the primary key of the **MEDICINE** entity, *MedicineID*, is added to the “*supplies*” relation renamed *mID*. Also, the primary key of **SELF_CARE_HYGIENE_PRODUCT**, *ProductID*, is added to the “*supplies*” relation renamed *prID*. The combination of all added keys is now the primary key of the newly added relationship “*supplies*”, thus being underlined.

2. Includes

<u>bID</u>	<u>mID</u>	<u>prID</u>
------------	------------	-------------

Many billing invoices include many medicines and self-care/ hygiene product. The “*includes*” relationship links the three entities **BILLING_INVOICE**, **MEDICINE**, and **SELF_CARE_HYGIENE_PRODUCT**. We create a new relation called “*includes*” in which we include the primary keys of the three entities **BILLING_INVOICE**, **MEDICINE**, and **SELF_CARE_HYGIENE_PRODUCT**. The primary key of the **BILLING_INVOICE** entity, *InvoiceID*, is added to the “*includes*” relation renamed *bID*. Then, the primary key of the **MEDICINE** entity, *MedicineID*, is added to the “*includes*” relation renamed *mID*. Also, the primary key of the **SELF_CARE_HYGIENE_PRODUCT** entity, *ProductID*, is added to the “*includes*” relation renamed *prID*. The combination of the three added keys now represents the primary key of the newly added relation “*includes*”, thus being underlined.

3. Stores

<u>mID</u>	<u>prID</u>	<u>inID</u>	Restock_date	Expiry
------------	-------------	-------------	--------------	--------

Many inventories store many medicines and self-care/hygiene products. The “stores” relationship links the three entities **INVENTORY**, **MEDICINE**, and **SELF_CARE_HYGIENE_PRODUCT**. We create a new relation called “stores” in which we include the primary keys of the three entities **INVENTORY**, **MEDICINE**, and **SELF_CARE_HYGIENE_PRODUCT**. The primary key of **INVENTORY** entity, InventoryID, is added to the “stores”, relation renamed inID. Then the primary key of the **MEDICINE** entity, MedicineID, is added to the “stores” relation, renamed mID. Also, the primary key of the **SELF_CARE_HYGIENE_PRODUCT** entity, ProductID, is added to the “stores” relation, renamed prID. Combining the three added keys now represents the primary key of the newly added relation “stores”, thus being underlined. Also, we didn’t forget the simple attributes of the “stores” relation: *Expiry* and *Restock_date*.

FINAL STEP: Final Displays

1. EMPLOYEE

<u>SSN</u>	Fname	Lname	Dob	Sex
Position	License_number	Salary	Phone_number	Email
City	Street_nb	Street_name	Apartment_number	B_ID
End_date	Start_date	Hours_worked		

2. BRANCH

<u>BranchID</u>	Name	Phone	Email	Street
City	Manager_name	Manager_SSN		

3. INVENTORY

<u>InventoryID</u>	Quantity	Street	City	B_ID
--------------------	----------	--------	------	------

4. SUPPLIER

<u>SupplierID</u>	Name	Contact_person	Delivery_term	Email
Phone_number	Street	City		

5. SELF_CARE_HYGIENE_PRODUCT

<u>ProductID</u>	Name	Description	Category	Price
Manufacturer				

6. MEDICINE

<u>MedicineID</u>	Name	Description	Prescription_required	Price
Manufacturer				

7. PRESCRIPTION

<u>PrescriptionID</u>	Date_issued	Refill_count	Expiry	Dr_phone_nb
Dr_name	Dr_license_number	Refills_frequency	CF_name	CL_name

8. BILLING_INVOICE

<u>InvoiceID</u>	ESSN	Date_issued	Total_amount	Payment_status
CF_name	CL_name	Amount_paid		

9. CUSTOMER

<u>Fname</u>	<u>Lname</u>	<u>Bill_ID</u>	Phone_number	Dob
--------------	--------------	----------------	--------------	-----

10.DEPENDENT

<u>Fname</u>	<u>Lname</u>	<u>ESSN</u>	Sex	Dob
Relationship				

11.contains2

<u>pID</u>	<u>mID</u>
------------	------------

12.Payment_method

<u>InvoiceID</u>	<u>Payment_method</u>
------------------	-----------------------

13.Item_ID

<u>InvoiceID</u>	<u>ItemID</u>
------------------	---------------

14.Dosage_instructions

<u>PrescriptionID</u>	<u>Dosage_instruction</u>
-----------------------	---------------------------

15.Payment_term

<u>SupplierID</u>	<u>Payment_term</u>
-------------------	---------------------

16.Supplies

<u>mID</u>	<u>sID</u>	<u>prID</u>	Price_per_unit	Supply_date
Quantity_supplied				

17.Includes

<u>bID</u>	<u>mID</u>	<u>prID</u>
------------	------------	-------------

18.Stores

<u>mID</u>	<u>prID</u>	<u>inID</u>	Restock_date	Expiry
------------	-------------	-------------	--------------	--------

Table Structure for Healer Dealer Pharmacy

We have now completed both the conceptual design of our Healer Dealer Pharmacy database and its mapping to the relational model. These tables will allow us to transition into the Oracle database server smoothly as we create the actual tables of our database. After that, we will start inserting values or tuples into these tables and finally add some sample transactions we could perform to display the importance of our database.

CREATE TABLES:

To create our tables in the Oracle database server, we must use the CREATE TABLE command and include the name of our relation and its attributes, as stated in the relational model, with their data types. Then, we include our primary key and any foreign keys or constraints if present.

1- BRANCH:

```
CREATE TABLE BRANCH (  
  BranchID VARCHAR(5) PRIMARY KEY,  
  Name VARCHAR(50) NOT NULL,  
  Phone VARCHAR(15),  
  Email VARCHAR(100) CHECK (Email LIKE '%@healerdealer.com'),  
  Street VARCHAR(100),  
  City VARCHAR(50),  
  Manager_name VARCHAR(50),  
  Manager_SSN VARCHAR(9)  
);
```

--Note: “**Manager_SSN**” is a foreign key referencing the primary key of **EMPLOYEE**. However, we face a certain issue of circular dependency, where **EMPLOYEE** also has a foreign key “**B_ID**” referencing the

primary key of **BRANCH**. To tackle this issue, we have decided not to state “**Manager_SSN**” as a foreign key during table creation, but we will use the ALTER command to add it later.

2- EMPLOYEE:

```
CREATE TABLE EMPLOYEE (  
    SSN VARCHAR(9) NOT NULL,  
    Fname VARCHAR(15) NOT NULL,  
    Lname VARCHAR(15) NOT NULL,  
    Dob DATE,  
    Sex CHAR NOT NULL CHECK(Sex IN ('F', 'M')),  
    Position VARCHAR(15),  
    License_number VARCHAR(20) DEFAULT NULL,  
    Salary DECIMAL(10, 2),  
    Phone_number VARCHAR(15) NOT NULL,  
    Email VARCHAR(40) CHECK (Email LIKE '%@healerdealer.com'),  
    City VARCHAR(30),  
    Street_nb INT,  
    Street_name VARCHAR(30),  
    Apartment_number INT,  
    B_ID VARCHAR(5),  
    Start_date DATE,  
    End_date DATE,  
    Hours_worked INT,  
    PRIMARY KEY (SSN),  
    FOREIGN KEY (B_ID) REFERENCES BRANCH(BranchID)  
);
```

3- SUPPLIER:

```
CREATE TABLE SUPPLIER (  
    SupplierID CHAR(11) NOT NULL PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Contact_person VARCHAR(50),  
    Delivery_term VARCHAR(30) CHECK(Delivery_term IN ('Weekly', 'Monthly', 'Yearly')),  
    Email VARCHAR(40) NOT NULL,  
    Phone_number VARCHAR(15),  
    Street VARCHAR(30),
```

```
City VARCHAR(30) NOT NULL  
);
```

4- BILLING_INVOICE:

```
CREATE TABLE BILLING_INVOICE (  
    InvoiceID INT PRIMARY KEY,  
    ESSN VARCHAR(9) NOT NULL,  
    Date_issued DATE NOT NULL,  
    Total_amount NUMBER(10, 2) NOT NULL,  
    Payment_status VARCHAR(20) NOT NULL,  
    CF_name VARCHAR(15) NOT NULL,  
    CL_name VARCHAR(15) NOT NULL,  
    Amount_paid NUMBER(10, 2) NOT NULL,  
    FOREIGN KEY (ESSN) REFERENCES EMPLOYEE(SSN)  
);
```

--Note: In this table, “**CF_name**” and “**CL_name**” are foreign keys that refer to the primary keys of **CUSTOMER**, considering they are unique. However, due to the circular dependency issue mentioned earlier, which the Oracle server cannot bypass, we omitted this constraint from the CREATE TABLE command. Instead, we added it later using the ALTER command, as shown below.

5- INVENTORY:

```
CREATE TABLE INVENTORY (  
    InventoryID CHAR(5) PRIMARY KEY,  
    Quantity INT NOT NULL CHECK (Quantity >= 0),  
    Street VARCHAR(100),  
    City VARCHAR(50),  
    B_ID VARCHAR(5) NOT NULL,  
    FOREIGN KEY (B_ID) REFERENCES BRANCH(BranchID)  
);
```

6- CUSTOMER:

```
CREATE TABLE CUSTOMER (  
    Fname VARCHAR(30),  
    Lname VARCHAR(30),
```

```

Bill_ID INT,
Phone_number VARCHAR(15),
Dob DATE,
PRIMARY KEY (Fname, Lname, Bill_ID),
CONSTRAINT unique_fname_lname UNIQUE (Fname, Lname),
FOREIGN KEY (Bill_ID) REFERENCES BILLING_INVOICE(InvoiceID)
);

```

7- MEDICINE:

```

CREATE TABLE MEDICINE (
    MedicineID VARCHAR(7) PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Description VARCHAR(100),
    Prescription_required CHAR(1) CHECK (Prescription_required IN ('T', 'F')) NOT NULL,
    Price DECIMAL(10, 2) NOT NULL,
    Manufacturer VARCHAR(100)
);

```

8- SELF_CARE_HYGIENE_PRODUCT:

```

CREATE TABLE SELF_CARE_HYGIENE_PRODUCT (
    ProductID VARCHAR(20) PRIMARY KEY,
    Name VARCHAR(25) NOT NULL,
    Description VARCHAR(100),
    Category VARCHAR(50),
    Price DECIMAL(10, 2) NOT NULL,
    Manufacturer VARCHAR(100)
);

```

9- PRESCRIPTION:

```

CREATE TABLE PRESCRIPTION (
    PrescriptionID INT PRIMARY KEY,
    Date_issued DATE NOT NULL,
    Refill_count INT DEFAULT 0,
    Expiry DATE NOT NULL,
    Dr_phone_nb VARCHAR(15),
    Dr_name VARCHAR(100),
    Dr_license_number VARCHAR(50),
    Refills_frequency INT,
    CF_name VARCHAR(15) NOT NULL,
    CL_name VARCHAR(15) NOT NULL,
    FOREIGN KEY (CF_name, CL_name) REFERENCES CUSTOMER(Fname, Lname)
);

```

10- DEPENDENT:

```
CREATE TABLE DEPENDENT (  
    Fname VARCHAR(30),  
    Lname VARCHAR(30),  
    ESSN VARCHAR(9) NOT NULL,  
    Sex CHAR NOT NULL CHECK (Sex IN ('F', 'M')),  
    Dob DATE,  
    Relationship VARCHAR(20) NOT NULL CHECK (Relationship IN ('Spouse','Child','Parent','other')),  
    PRIMARY KEY (Fname, Lname, ESSN),  
    FOREIGN KEY (ESSN) REFERENCES EMPLOYEE(SSN)  
);
```

11- Payment_term:

```
CREATE TABLE Payment_term (  
    SupplierID CHAR(11) NOT NULL,  
    Payment_term VARCHAR(30) NOT NULL,  
    PRIMARY KEY (SupplierID, Payment_term),  
    FOREIGN KEY (SupplierID) REFERENCES SUPPLIER(SupplierID)  
);
```

12- contains2:

```
CREATE TABLE contains2 (  
    pID INT NOT NULL,  
    mID VARCHAR(7),  
    PRIMARY KEY (pID, mID),  
    FOREIGN KEY (mID) REFERENCES MEDICINE(MedicineID),  
    FOREIGN KEY (pID) REFERENCES PRESCRIPTION(PrescriptionID)  
);
```

13- Payment_method:

```
CREATE TABLE Payment_method (  
    InvoiceID INT NOT NULL,  
    Payment_method VARCHAR(20),  
    PRIMARY KEY (InvoiceID, Payment_method),
```

```
FOREIGN KEY (InvoiceID) REFERENCES BILLING_INVOICE(InvoiceID)
);
```

14- Item_ID:

```
CREATE TABLE Item_ID (
    InvoiceID INT NOT NULL,
    ItemID VARCHAR(10),
    PRIMARY KEY (InvoiceID, ItemID),
    FOREIGN KEY (InvoiceID) REFERENCES BILLING_INVOICE(InvoiceID)
);
```

15- Dosage_Instructions:

```
CREATE TABLE Dosage_Instructions (
    PrescriptionID INT NOT NULL,
    Dosage_instruction VARCHAR(100),
    PRIMARY KEY (PrescriptionID, Dosage_instruction),
    FOREIGN KEY (PrescriptionID) REFERENCES PRESCRIPTION(PrescriptionID)
);
```

16- Supplies:

```
CREATE TABLE Supplies (
    mID VARCHAR(7),
    sID CHAR(11) NOT NULL,
    prID VARCHAR(20),
    Price_per_unit DECIMAL(10, 2) NOT NULL,
    Supply_date DATE NOT NULL,
    Quantity_supplied INT NOT NULL,
    PRIMARY KEY (mID, sID, prID),
    FOREIGN KEY (mID) REFERENCES MEDICINE(MedicineID),
    FOREIGN KEY (prID) REFERENCES SELF_CARE_HYGIENE_PRODUCT(ProductID),
    FOREIGN KEY (sID) REFERENCES SUPPLIER(SupplierID)
);
```

17- Includes:

```
CREATE TABLE Includes (
    bID INT NOT NULL,
    mID VARCHAR(7),
    prID VARCHAR(20),
    PRIMARY KEY (bID, mID, prID),
```

```

FOREIGN KEY (bID) REFERENCES BILLING_INVOICE(InvoiceID),
FOREIGN KEY (mID) REFERENCES MEDICINE(MedicineID),
FOREIGN KEY (prID) REFERENCES SELF_CARE_HYGIENE_PRODUCT(ProductID)
);

```

18- Stores:

```

CREATE TABLE Stores (
  mID VARCHAR(7),
  prID VARCHAR(20),
  inID CHAR(5) NOT NULL,
  Restock_date DATE,
  Expiry DATE NOT NULL,
  PRIMARY KEY (mID, prID, inID),
  FOREIGN KEY (mID) REFERENCES MEDICINE(MedicineID),
  FOREIGN KEY (prID) REFERENCES SELF_CARE_HYGIENE_PRODUCT(ProductID),
  FOREIGN KEY (inID) REFERENCES INVENTORY(InventoryID)
);

```

ALTER command:

As written in the previous notes, some referential integrity constraints (foreign keys) cannot be added directly upon creation of the table because the primary key it references is still not created, and due to the circular dependency issue with both tables referencing each other, this couldn't be solved with a simple re-ordering. We are obliged to use the ALTER command:

```
ALTER TABLE BRANCH ADD (FOREIGN KEY (Manager_SSN) REFERENCES EMPLOYEE(SSN));
```

```
ALTER TABLE BILLING_INVOICE ADD( FOREIGN KEY (CF_name, CL_name) REFERENCES
CUSTOMER(Fname, Lname));
```

Table Descriptions:

1-BRANCH:

Query: DESC BRANCH;

Output:

Object Type **TABLE** Object **BRANCH**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>BRANCH</u>	<u>BRANCHID</u>	VARCHAR2	5	-	-	1	-	-	-
	<u>NAME</u>	VARCHAR2	50	-	-	-	-	-	-
	<u>PHONE</u>	VARCHAR2	15	-	-	-	✓	-	-
	<u>EMAIL</u>	VARCHAR2	100	-	-	-	✓	-	-
	<u>STREET</u>	VARCHAR2	100	-	-	-	✓	-	-
	<u>CITY</u>	VARCHAR2	50	-	-	-	✓	-	-
	<u>MANAGER_NAME</u>	VARCHAR2	50	-	-	-	✓	-	-
	<u>MANAGER_SSN</u>	VARCHAR2	9	-	-	-	✓	-	-
1 - 8									

2- EMPLOYEE:

Query: DESC EMPLOYEE;

Output:

Object Type **TABLE** Object **EMPLOYEE**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>EMPLOYEE</u>	<u>SSN</u>	VARCHAR2	9	-	-	1	-	-	-
	<u>FNAME</u>	VARCHAR2	15	-	-	-	-	-	-
	<u>LNAME</u>	VARCHAR2	15	-	-	-	-	-	-
	<u>DOB</u>	DATE	7	-	-	-	✓	-	-
	<u>SEX</u>	CHAR	1	-	-	-	-	-	-
	<u>POSITION</u>	VARCHAR2	15	-	-	-	✓	-	-
	<u>LICENSE_NUMBER</u>	VARCHAR2	20	-	-	-	✓	NULL	-
	<u>SALARY</u>	NUMBER	-	10	2	-	✓	-	-
	<u>PHONE_NUMBER</u>	VARCHAR2	15	-	-	-	-	-	-
	<u>EMAIL</u>	VARCHAR2	40	-	-	-	✓	-	-
	<u>CITY</u>	VARCHAR2	30	-	-	-	✓	-	-
	<u>STREET_NB</u>	NUMBER	22	-	0	-	✓	-	-
	<u>STREET_NAME</u>	VARCHAR2	30	-	-	-	✓	-	-
	<u>APARTMENT_NUMBER</u>	NUMBER	22	-	0	-	✓	-	-
	<u>B_ID</u>	VARCHAR2	5	-	-	-	✓	-	-
	<u>START_DATE</u>	DATE	7	-	-	-	✓	-	-
	<u>END_DATE</u>	DATE	7	-	-	-	✓	-	-
	<u>HOURS_WORKED</u>	NUMBER	22	-	0	-	✓	-	-
1 - 18									

3-SUPPLIER:

Query: DESC SUPPLIER;

Output:

Object Type **TABLE** Object **SUPPLIER**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>SUPPLIER</u>	<u>SUPPLIERID</u>	CHAR	11	-	-	1	-	-	-
	<u>NAME</u>	VARCHAR2	100	-	-	-	-	-	-
	<u>CONTACT_PERSON</u>	VARCHAR2	50	-	-	-	✓	-	-
	<u>DELIVERY_TERM</u>	VARCHAR2	30	-	-	-	✓	-	-
	<u>EMAIL</u>	VARCHAR2	40	-	-	-	-	-	-
	<u>PHONE_NUMBER</u>	VARCHAR2	15	-	-	-	✓	-	-
	<u>STREET</u>	VARCHAR2	30	-	-	-	✓	-	-
	<u>CITY</u>	VARCHAR2	30	-	-	-	-	-	-
									1 - 8

4- BILLING_INVOICE:

Query: DESC BILLING_INVOICE;

Output:

Object Type **TABLE** Object **BILLING_INVOICE**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>BILLING_INVOICE</u>	<u>INVOICEID</u>	NUMBER	22	-	0	1	-	-	-
	<u>ESSN</u>	VARCHAR2	9	-	-	-	-	-	-
	<u>DATE_ISSUED</u>	DATE	7	-	-	-	-	-	-
	<u>TOTAL_AMOUNT</u>	NUMBER	-	10	2	-	-	-	-
	<u>PAYMENT_STATUS</u>	VARCHAR2	20	-	-	-	-	-	-
	<u>CF_NAME</u>	VARCHAR2	15	-	-	-	-	-	-
	<u>CL_NAME</u>	VARCHAR2	15	-	-	-	-	-	-
	<u>AMOUNT_PAID</u>	NUMBER	-	10	2	-	-	-	-
									1 - 8

5- INVENTORY:

Query: DESC INVENTORY;

Output:

Object Type **TABLE** Object **INVENTORY**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>INVENTORY</u>	<u>INVENTORYID</u>	CHAR	5	-	-	1	-	-	-
	<u>QUANTITY</u>	NUMBER	22	-	0	-	-	-	-
	<u>STREET</u>	VARCHAR2	100	-	-	-	✓	-	-
	<u>CITY</u>	VARCHAR2	50	-	-	-	✓	-	-
	<u>B_ID</u>	VARCHAR2	5	-	-	-	-	-	-
									1 - 5

6- CUSTOMER

Query: DESC CUSTOMER;
Output:

Object Type TABLE Object CUSTOMER									
Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CUSTOMER	FNAME	VARCHAR2	30	-	-	1	-	-	-
	LNAME	VARCHAR2	30	-	-	2	-	-	-
	BILL_ID	NUMBER	22	-	0	3	-	-	-
	PHONE_NUMBER	VARCHAR2	15	-	-	-	✓	-	-
	DOB	DATE	7	-	-	-	✓	-	-
									1 - 5

7- MEDICINE:

Query: DESC MEDICINE;
Output:

Object Type TABLE Object MEDICINE									
Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
MEDICINE	MEDICINEID	VARCHAR2	7	-	-	1	-	-	-
	NAME	VARCHAR2	50	-	-	-	-	-	-
	DESCRIPTION	VARCHAR2	100	-	-	-	✓	-	-
	PRESCRIPTION_REQUIRED	CHAR	1	-	-	-	-	-	-
	PRICE	NUMBER	-	10	2	-	-	-	-
	MANUFACTURER	VARCHAR2	100	-	-	-	✓	-	-
									1 - 6

8- SELF_CARE_HYGIENE_PRODUCT:

Query: DESC SELF_CARE_HYGIENE_PRODUCT;
Output:

Object Type TABLE Object SELF_CARE_HYGIENE_PRODUCT									
Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
SELF_CARE_HYGIENE_PRODUCT	PRODUCTID	VARCHAR2	20	-	-	1	-	-	-
	NAME	VARCHAR2	25	-	-	-	-	-	-
	DESCRIPTION	VARCHAR2	100	-	-	-	✓	-	-
	CATEGORY	VARCHAR2	50	-	-	-	✓	-	-
	PRICE	NUMBER	-	10	2	-	-	-	-
	MANUFACTURER	VARCHAR2	100	-	-	-	✓	-	-
									1 - 6

9- PRESCRIPTION:

Query: DESC PRESCRIPTION;
Output:

Object Type TABLE Object PRESCRIPTION									
Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>PRESCRIPTION</u>	<u>PRESCRIPTIONID</u>	NUMBER	22	-	0	1	-	-	-
	<u>DATE_ISSUED</u>	DATE	7	-	-	-	-	-	-
	<u>REFILL_COUNT</u>	NUMBER	22	-	0	-	✓	0	-
	<u>EXPIRY</u>	DATE	7	-	-	-	-	-	-
	<u>DR_PHONE_NB</u>	VARCHAR2	15	-	-	-	✓	-	-
	<u>DR_NAME</u>	VARCHAR2	100	-	-	-	✓	-	-
	<u>DR_LICENSE_NUMBER</u>	VARCHAR2	50	-	-	-	✓	-	-
	<u>REFILLS_FREQUENCY</u>	NUMBER	22	-	0	-	✓	-	-
	<u>CF_NAME</u>	VARCHAR2	15	-	-	-	-	-	-
	<u>CL_NAME</u>	VARCHAR2	15	-	-	-	-	-	-
									1 - 10

10- DEPENDENT

Query: DESC DEPENDENT;
Output:

Object Type **TABLE** Object **DEPENDENT**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>DEPENDENT</u>	<u>FNAME</u>	VARCHAR2	30	-	-	1	-	-	-
	<u>LNAME</u>	VARCHAR2	30	-	-	2	-	-	-
	<u>ESSN</u>	VARCHAR2	9	-	-	3	-	-	-
	<u>SEX</u>	CHAR	1	-	-	-	-	-	-
	<u>DOB</u>	DATE	7	-	-	-	✓	-	-
	<u>RELATIONSHIP</u>	VARCHAR2	20	-	-	-	-	-	-
									1 - 6

11- PAYMENT_TERM

Query: DESC PAYMENT_TERM;

Output:

Object Type **TABLE** Object **PAYMENT_TERM**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>PAYMENT_TERM</u>	<u>SUPPLIERID</u>	CHAR	11	-	-	1	-	-	-
	<u>PAYMENT_TERM</u>	VARCHAR2	30	-	-	2	-	-	-
									1 - 2

12- CONTAINS2

Query: DESC CONTAINS2;

Output:

Object Type **TABLE** Object **CONTAINS2**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>CONTAINS2</u>	<u>PID</u>	NUMBER	22	-	0	1	-	-	-
	<u>MID</u>	VARCHAR2	7	-	-	2	-	-	-
									1 - 2

13- PAYMENT_METHOD

Query: DESC PAYMENT_METHOD;

Output:

Object Type **TABLE** Object **PAYMENT_METHOD**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>PAYMENT_METHOD</u>	<u>INVOICEID</u>	NUMBER	22	-	0	1	-	-	-
	<u>PAYMENT_METHOD</u>	VARCHAR2	20	-	-	2	-	-	-
									1 - 2

14- ITEM_ID

Query: DESC ITEM_ID;

Output:

Object Type		TABLE Object		ITEM_ID						
Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment	
ITEM_ID	INVOICEID	NUMBER	22	-	0	1	-	-	-	
	ITEMID	VARCHAR2	10	-	-	2	-	-	-	
										1 - 2

15- DOSAGE_INSTRUCTIONS

Query: DESC DOSAGE_INSTRUCTIONS;

Output:

Object Type		TABLE Object		DOSAGE_INSTRUCTIONS						
Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment	
DOSAGE_INSTRUCTIONS	PRESCRIPTIONID	NUMBER	22	-	0	1	-	-	-	
	DOSAGE_INSTRUCTION	VARCHAR2	100	-	-	2	-	-	-	
										1 - 2

16- SUPPLIES

Query: DESC SUPPLIES;

Output:

Object Type		TABLE Object		SUPPLIES						
Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment	
SUPPLIES	MID	VARCHAR2	7	-	-	1	-	-	-	
	SID	CHAR	11	-	-	2	-	-	-	
	PRID	VARCHAR2	20	-	-	3	-	-	-	
	PRICE_PER_UNIT	NUMBER	-	10	2	-	-	-	-	
	SUPPLY_DATE	DATE	7	-	-	-	-	-	-	
	QUANTITY_SUPPLIED	NUMBER	22	-	0	-	-	-	-	
										1 - 6

17- INCLUDES

Query: DESC INCLUDES;

Output:

Object Type **TABLE** Object **INCLUDES**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>INCLUDES</u>	<u>BID</u>	NUMBER	22	-	0	1	-	-	-
	<u>MID</u>	VARCHAR2	7	-	-	2	-	-	-
	<u>PRID</u>	VARCHAR2	20	-	-	3	-	-	-
									1 - 3

18- STORES;

Query: DESC STORES;

Output:

Object Type **TABLE** Object **STORES**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>STORES</u>	<u>MID</u>	VARCHAR2	7	-	-	1	-	-	-
	<u>PRID</u>	VARCHAR2	20	-	-	2	-	-	-
	<u>INID</u>	CHAR	5	-	-	3	-	-	-
	<u>RESTOCK_DATE</u>	DATE	7	-	-	-	✓	-	-
	<u>EXPIRY</u>	DATE	7	-	-	-	-	-	-
									1 - 5

TABLE STATE

Inserting Values:

1. BRANCH:

INSERT INTO BRANCH VALUES ('B001', 'Hamra HD Branch', '+96119876543', 'hamra@healerdealer.com', 'Hamra Street', 'Beirut', 'Rami Haddad', NULL);

INSERT INTO BRANCH VALUES ('B002', 'Bliss HD Branch', '+96119987654', 'bliss@healerdealer.com', 'Bliss St', 'Beirut', 'Hassan Amin', NULL);

INSERT INTO BRANCH VALUES ('B003', 'Achrafieh HD Branch', '+96117123456', 'achrafieh@healerdealer.com', 'Achrafieh St', 'Beirut', 'Samir Khalil', NULL);

2. EMPLOYEE:

```
INSERT INTO EMPLOYEE VALUES ('123456789', 'Rami', 'Haddad', TO_DATE('12/02/1985',
'DD/MM/YYYY'), 'M', 'Manager', NULL, 2000.00, '+96112345678', 'rami.haddad@healerdealer.com',
'Beirut', 1, 'Hamra St', 3, 'B001', TO_DATE('01/01/2022', 'DD/MM/YYYY'), NULL, 40);
```

```
INSERT INTO EMPLOYEE VALUES ('987654321', 'Noura', 'Elia', TO_DATE('05/06/1990',
'DD/MM/YYYY'), 'F', 'Cashier', NULL, 1200.00, '+96176345678', 'noura.elia@healerdealer.com',
'Beirut', 5, 'Verdan St', 5, 'B002', TO_DATE('15/07/2022', 'DD/MM/YYYY'), NULL, 35);
```

```
INSERT INTO EMPLOYEE VALUES ('112233445', 'Fadi', 'Kassem', TO_DATE('23/08/1980',
'DD/MM/YYYY'), 'M', 'Accountant', NULL, 2500.00, '+96171234567', 'fadi.kassem@healerdealer.com',
'Beirut', 10, 'Achrafieh St', 3, 'B003', TO_DATE('01/02/2023', 'DD/MM/YYYY'), NULL, 45);
```

```
INSERT INTO EMPLOYEE VALUES ('223344556', 'Layla', 'Zein', TO_DATE('16/12/1987',
'DD/MM/YYYY'), 'F', 'Pharmacist', 'PH654321', 1800.00, '+96174567890',
'layla.zein@healerdealer.com', 'Beirut', 50, 'Karatina St', 4, 'B001', TO_DATE('05/03/2022',
'DD/MM/YYYY'), NULL, 40);
```

```
INSERT INTO EMPLOYEE VALUES ('334455667', 'Hassan', 'Amin', TO_DATE('25/09/1992',
'DD/MM/YYYY'), 'M', 'Manager', NULL, 3000.00, '+96180567890', 'hassan.amin@healerdealer.com',
'Beirut', 10, 'Verdan St', 5, 'B002', TO_DATE('01/07/2022', 'DD/MM/YYYY'), NULL, 50);
```

```
INSERT INTO EMPLOYEE VALUES ('445566778', 'Maya', 'Saad', TO_DATE('14/03/1991',
'DD/MM/YYYY'), 'F', 'Cashier', NULL, 1100.00, '+96170654321', 'maya.saad@healerdealer.com',
'Beirut', 75, 'Sin El Fil St', 6, 'B003', TO_DATE('11/08/2022', 'DD/MM/YYYY'), NULL, 30);
```

```
INSERT INTO EMPLOYEE VALUES ('556677889', 'Ali', 'Jaber', TO_DATE('03/05/1988',
'DD/MM/YYYY'), 'M', 'Cleaner', NULL, 800.00, '+96195543210', 'ali.jaber@healerdealer.com', 'Beirut',
1, 'Hamra St', 7, 'B001', TO_DATE('20/09/2022', 'DD/MM/YYYY'), NULL, 40);
```

```
INSERT INTO EMPLOYEE VALUES ('667788990', 'Rita', 'Daher', TO_DATE('20/07/1989',
'DD/MM/YYYY'), 'F', 'Pharmacist', 'PH789123', 2200.00, '+96191567890', 'rita.daher@healerdealer.com',
'Beirut', 10, 'Achrafieh St', 8, 'B002', TO_DATE('12/06/2022', 'DD/MM/YYYY'), NULL, 42);
```

```
INSERT INTO EMPLOYEE VALUES ('778899001', 'Samir', 'Khalil', TO_DATE('09/11/1993',
'DD/MM/YYYY'), 'M', 'Manager', NULL, 1000.00, '+96173567890', 'samir.khalil@healerdealer.com',
'Beirut', 75, 'Sin El Fil St', 9, 'B003', TO_DATE('19/05/2022', 'DD/MM/YYYY'), NULL, 38);
```

```
INSERT INTO EMPLOYEE VALUES ('889900112', 'Zeinab', 'Fakhoury', TO_DATE('02/01/1986',
'DD/MM/YYYY'), 'F', 'Accountant', NULL, 2700.00, '+96182567890',
'zeinab.fakhoury@healerdealer.com', 'Beirut', 30, 'Monot St', 10, 'B001', TO_DATE('09/08/2022',
'DD/MM/YYYY'), NULL, 45);
```

--To avoid the error with the referential integrity constraint caused by foreign key Manager_SSN having nothing to refer to, we inserted Manager_SSN as a NULL value.

--Then, we updated them to reference the primary key values of EMPLOYEE after inserting EMPLOYEE values:

```
UPDATE BRANCH
SET Manager_SSN = '123456789'
WHERE BranchID = 'B001';
```

```
UPDATE BRANCH
SET Manager_SSN = '334455667'
WHERE BranchID = 'B002';
```

```
UPDATE BRANCH
SET Manager_SSN = '778899001'
WHERE BranchID = 'B003';
```

3. INVENTORY:

```
INSERT INTO INVENTORY VALUES ('I001', 100, 'Hamra St', 'Beirut', 'B001');
```

```
INSERT INTO INVENTORY VALUES ('I002', 200, 'Bliss St', 'Beirut', 'B002');
```

```
INSERT INTO INVENTORY VALUES ('I003', 150, 'Achrafieh St', 'Beirut', 'B003');
```

4. SUPPLIER:

```
INSERT INTO SUPPLIER VALUES ('30012345678', 'Beirut Pharma Solutions', 'Joseph Atallah',
'Monthly', 'contact@pharmaleb.com', '+96130123456', 'Jal el-Dib Street', 'Metn');
```

```
INSERT INTO SUPPLIER VALUES ('30087654321', 'Cedars Medical Supply Co.', 'Samir Bassil',
'Weekly', 'info@medico.com', '+96130876543', 'Zalka Main Street', 'Metn');
```

```
INSERT INTO SUPPLIER VALUES ('30011223344', 'LebHealth Pharmaceuticals', 'Rania Daher',
'Yearly', 'sales@healthplus.com', '+96130456789', 'Chiah Street', 'Beirut');
```

```
INSERT INTO SUPPLIER VALUES ('30044556677', 'Mount Lebanon Drug Distributors', 'Kassem Faris',
'Weekly', 'medleben@med.com', '+96131234567', 'Maameltein Street', 'Jounieh');
```

```
INSERT INTO SUPPLIER VALUES ('30099887766', 'Phoenicia Medica', 'Yasmin Akram', 'Monthly',
'biopharma@bph.com', '+96133234567', 'Riad el-Solh Street', 'Beirut');
```

```
INSERT INTO SUPPLIER VALUES ('30022334455', 'MedCare Solutions Lebanon',
', Khaled Taha', 'Yearly', 'info@lebmed.com', '+96135098765', 'Beirut Souks Street', 'Beirut');
```

```
INSERT INTO SUPPLIER VALUES ('30055667788', 'Cedar Valley Pharma', 'Maya Shamas', 'Monthly',  
'contact@careplus.com', '+96134356789', 'Hamra Street', 'Beirut');
```

```
INSERT INTO SUPPLIER VALUES ('30066778899', 'Lebanese Life Sciences Supply',  
, 'Rami Saba', 'Weekly', 'info@healthcareco.com', '+96136457890', 'City Centre', 'Beirut');
```

```
INSERT INTO SUPPLIER VALUES ('30077889900', 'Sahha Pharma Distributors', 'Zahra Harb',  
'Weekly', 'sales@medicare.com', '+96137548901', 'Ain Mreisseh Street', 'Beirut');
```

```
INSERT INTO SUPPLIER VALUES ('30088990011', 'AlphaMed Lebanon', 'Jad Khoury', 'Monthly',  
'contact@pharmamed.com', '+96138765012', 'Rue de Verdun', 'Beirut');
```

5. CUSTOMER:

Since CUSTOMER and BILLING_INVOICE have foreign keys that refer to each other, there are a few steps extra we must follow, as with EMPLOYEE and BRANCH:

--We first get the constraint name that prevents primary key Bill_ID from being NULL, and drop it later to make Bill_ID nullable:

```
SELECT constraint_name  
FROM user_constraints  
WHERE table_name = 'CUSTOMER';
```

--Drop primary key constraint to make bill_id nullable:

```
ALTER TABLE CUSTOMER DROP CONSTRAINT SYS_C007488;  
ALTER TABLE CUSTOMER MODIFY Bill_ID INT NULL;
```

--Then insert CUSTOMER values with null Bill_ID:

```
INSERT INTO CUSTOMER VALUES ('Sami', 'Haddad', NULL, '+9611111111',  
TO_DATE('05/05/1995', 'DD/MM/YYYY'));
```

```
INSERT INTO CUSTOMER VALUES ('Rita', 'Khalil', NULL, '+9612222222', TO_DATE('10/10/1985',  
'DD/MM/YYYY'));
```

```
INSERT INTO CUSTOMER VALUES ('Maya', 'Saad', NULL, '+9613333333', TO_DATE('08/02/1990',  
'DD/MM/YYYY'));
```

```
INSERT INTO CUSTOMER VALUES ('Ali', 'Jaber', NULL, '+9614444444', TO_DATE('14/03/1987',  
'DD/MM/YYYY'));
```

```
INSERT INTO CUSTOMER VALUES ('Zeinab', 'Fakhoury', NULL, '+9615555555',  
TO_DATE('20/04/1992', 'DD/MM/YYYY'));
```

```
INSERT INTO CUSTOMER VALUES ('Fadi', 'Kassem', NULL, '+96166666666',  
TO_DATE('12/07/1980', 'DD/MM/YYYY'));
```

```
INSERT INTO CUSTOMER VALUES ('Samir', 'Khalil', NULL, '+96177777777',  
TO_DATE('22/09/1989', 'DD/MM/YYYY'));
```

```
INSERT INTO CUSTOMER VALUES ('Hassan', 'Amin', NULL, '+96188888888',  
TO_DATE('05/06/1993', 'DD/MM/YYYY'));
```

```
INSERT INTO CUSTOMER VALUES ('Layla', 'Zein', NULL, '+96199999999', TO_DATE('13/08/1986',  
'DD/MM/YYYY'));
```

```
INSERT INTO CUSTOMER VALUES ('Rami', 'Haddad', NULL, '+96110000000',  
TO_DATE('17/11/1984', 'DD/MM/YYYY'));
```

6. BILLING_INVOICE:

--Then we insert BILLING_INVOICE values:

```
INSERT INTO BILLING_INVOICE VALUES (1, '123456789', TO_DATE('02/11/2024',  
'DD/MM/YYYY'), 15.00, 'paid', 'Sami', 'Haddad', 15.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (2, '987654321', TO_DATE('16/11/2024',  
'DD/MM/YYYY'), 20.00, 'pending', 'Rita', 'Khalil', 10.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (3, '112233445', TO_DATE('19/11/2024',  
'DD/MM/YYYY'), 30.00, 'paid', 'Maya', 'Saad', 30.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (4, '223344556', TO_DATE('21/11/2024',  
'DD/MM/YYYY'), 40.00, 'pending', 'Ali', 'Jaber', 10.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (5, '334455667', TO_DATE('23/11/2024',  
'DD/MM/YYYY'), 50.00, 'paid', 'Fadi', 'Kassem', 50.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (6, '445566778', TO_DATE('25/11/2024',  
'DD/MM/YYYY'), 60.00, 'pending', 'Samir', 'Khalil', 30.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (7, '556677889', TO_DATE('26/11/2024',  
'DD/MM/YYYY'), 70.00, 'paid', 'Zeinab', 'Fakhoury', 70.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (8, '667788990', TO_DATE('28/11/2024',  
'DD/MM/YYYY'), 80.00, 'pending', 'Layla', 'Zein', 40.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (9, '778899001', TO_DATE('30/11/2024',  
'DD/MM/YYYY'), 90.00, 'paid', 'Hassan', 'Amin', 90.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (10, '889900112', TO_DATE('01/12/2024',  
'DD/MM/YYYY'), 100.00, 'pending', 'Rami', 'Haddad', 50.00);
```

--Now that we added BILLING_INVOICE values, we have the primary key InvoiceID to refer to in the foreign key of Bill_ID of CUSTOMER:

```
UPDATE CUSTOMER
SET Bill_ID = 1
WHERE Fname = 'Sami' AND Lname='Haddad';
```

```
UPDATE CUSTOMER
SET Bill_ID = 2
WHERE Fname = 'Rita' AND Lname='Khalil';
```

```
UPDATE CUSTOMER
SET Bill_ID = 3
WHERE Fname = 'Maya' AND Lname='Saad';
```

```
UPDATE CUSTOMER
SET Bill_ID = 4
WHERE Fname = 'Ali' AND Lname='Jaber';
```

```
UPDATE CUSTOMER
SET Bill_ID = 5
WHERE Fname = 'Fadi' AND Lname='Kassem';
```

```
UPDATE CUSTOMER
SET Bill_ID = 6
WHERE Fname = 'Samir' AND Lname='Khalil';
```

```
UPDATE CUSTOMER
SET Bill_ID = 7
WHERE Fname = 'Zeinab' AND Lname='Fakhoury';
```

```
UPDATE CUSTOMER
SET Bill_ID = 8
WHERE Fname = 'Layla' AND Lname='Zein';
```

```
UPDATE CUSTOMER
SET Bill_ID = 9
WHERE Fname = 'Hassan' AND Lname='Amin';
```

```
UPDATE CUSTOMER
SET Bill_ID = 10
WHERE Fname = 'Rami' AND Lname = 'Haddad';
```

--Finally, we re-establish the constraints of CUSTOMER:

```
ALTER TABLE CUSTOMER ADD CONSTRAINT PK_CUSTOMER PRIMARY KEY (Fname, Lname,
Bill_ID);
```

7. DEPENDENT:

INSERT INTO DEPENDENT VALUES ('Lila', 'Haddad', '123456789', 'F', TO_DATE('10/10/2012', 'DD/MM/YYYY'), 'Child');

INSERT INTO DEPENDENT VALUES ('Nadim', 'Haddad', '123456789', 'M', TO_DATE('15/05/2010', 'DD/MM/YYYY'), 'Child');

INSERT INTO DEPENDENT VALUES ('Maya', 'Amin', '334455667', 'F', TO_DATE('22/06/2015', 'DD/MM/YYYY'), 'Spouse');

INSERT INTO DEPENDENT VALUES ('Samir', 'Saad', '445566778', 'M', TO_DATE('30/09/2012', 'DD/MM/YYYY'), 'Child');

INSERT INTO DEPENDENT VALUES ('Tariq', 'Jaber', '556677889', 'M', TO_DATE('02/02/2014', 'DD/MM/YYYY'), 'Child');

INSERT INTO DEPENDENT VALUES ('Rania', 'Zein', '667788990', 'F', TO_DATE('10/11/2013', 'DD/MM/YYYY'), 'Spouse');

INSERT INTO DEPENDENT VALUES ('Layla', 'Saad', '778899001', 'F', TO_DATE('14/04/2013', 'DD/MM/YYYY'), 'Child');

INSERT INTO DEPENDENT VALUES ('Fadi', 'Kassem', '889900112', 'M', TO_DATE('25/03/2016', 'DD/MM/YYYY'), 'Child');

INSERT INTO DEPENDENT VALUES ('Zeinab', 'Fakhoury', '889900112', 'F', TO_DATE('28/07/2011', 'DD/MM/YYYY'), 'Child');

INSERT INTO DEPENDENT VALUES ('Rami', 'Haddad', '123456789', 'M', TO_DATE('02/05/2009', 'DD/MM/YYYY'), 'Spouse');

8. Payment_term:

INSERT INTO Payment_term VALUES ('30012345678', 'Monthly');

INSERT INTO Payment_term VALUES ('30087654321', 'Weekly');

INSERT INTO Payment_term VALUES ('30011223344', 'Yearly');

INSERT INTO Payment_term VALUES ('30044556677', 'Monthly');

INSERT INTO Payment_term VALUES ('30099887766', 'Weekly');

INSERT INTO Payment_term VALUES ('30022334455', 'Yearly');

INSERT INTO Payment_term VALUES ('30055667788', 'Monthly');

INSERT INTO Payment_term VALUES ('30066778899', 'Weekly');

INSERT INTO Payment_term VALUES ('30077889900', 'Monthly');

INSERT INTO Payment_term VALUES ('30088990011', 'Yearly');

9. MEDICINE:

INSERT INTO MEDICINE VALUES ('A01AA01', 'Paracetamol', 'Pain reliever', 'T', 5.00, 'PharmaLeb');

INSERT INTO MEDICINE VALUES ('B02BC01', 'Ibuprofen', 'Anti-inflammatory', 'T', 10.00, 'Medico');

INSERT INTO MEDICINE VALUES ('C03CC01', 'Aspirin', 'Blood thinner', 'T', 7.00, 'HealthPlus');

INSERT INTO MEDICINE VALUES ('D04DC01', 'Loratadine', 'Antihistamine', 'T', 8.50, 'PharmaLeb');

INSERT INTO MEDICINE VALUES ('E05EE01', 'Cetirizine', 'Antihistamine', 'T', 6.50, 'Medico');

INSERT INTO MEDICINE VALUES ('F06FF01', 'Hydrocortisone', 'Steroid', 'T', 12.00, 'HealthPlus');

INSERT INTO MEDICINE VALUES ('G07GG01', 'Ciprofloxacin', 'Antibiotic', 'T', 15.00, 'PharmaLeb');

INSERT INTO MEDICINE VALUES ('H08HH01', 'Clindamycin', 'Antibiotic', 'T', 13.00, 'Medico');

INSERT INTO MEDICINE VALUES ('I09II01', 'Amoxicillin', 'Antibiotic', 'T', 9.00, 'HealthPlus');

INSERT INTO MEDICINE VALUES ('J10JJ01', 'Omeprazole', 'Acid reducer', 'T', 10.50, 'PharmaLeb');

INSERT INTO MEDICINE VALUES ('N/A', 'N/A', 'N/A', 'F', 0, 'N/A');

--We inserted a 'N/A' tuple that represents “no medicine”. This is to account for instances like when the customer buys only products and no medicine, as we have some ternary relationships that merge them and use the products and medicine primary keys as the ternary relation’s composite key.

10. SELF_CARE_HYGIENE_PRODUCT:

INSERT INTO SELF_CARE_HYGIENE_PRODUCT VALUES ('SC1001', 'Toothpaste', 'Mint flavor', 'Dental', 3.00, 'OralB');

INSERT INTO SELF_CARE_HYGIENE_PRODUCT VALUES ('SC1002', 'Shampoo', 'For dry hair', 'Hair Care', 8.00, 'Dove');

```
INSERT INTO SELF_CARE_HYGIENE_PRODUCT VALUES ('SC1003', 'Body Lotion', 'Moisturizing',  
'Skin Care', 5.50, 'Nivea');
```

```
INSERT INTO SELF_CARE_HYGIENE_PRODUCT VALUES ('SC1004', 'Face Wash', 'For sensitive  
skin', 'Skin Care', 6.00, 'Neutrogena');
```

```
INSERT INTO SELF_CARE_HYGIENE_PRODUCT VALUES ('SC1005', 'Conditioner', 'For curly hair',  
'Hair Care', 7.00, 'Pantene');
```

```
INSERT INTO SELF_CARE_HYGIENE_PRODUCT VALUES ('SC1006', 'Hand Soap', 'Lavender  
scent', 'Personal Care', 2.50, 'Lifebuoy');
```

```
INSERT INTO SELF_CARE_HYGIENE_PRODUCT VALUES ('SC1007', 'Shaving Cream', 'Sensitive  
skin', 'Personal Care', 4.00, 'Gillette');
```

```
INSERT INTO SELF_CARE_HYGIENE_PRODUCT VALUES ('SC1008', 'Shower Gel', 'For men', 'Skin  
Care', 6.50, 'Old Spice');
```

```
INSERT INTO SELF_CARE_HYGIENE_PRODUCT VALUES ('SC1009', 'Lip Balm', 'Nourishing',  
'Personal Care', 2.00, 'ChapStick');
```

```
INSERT INTO SELF_CARE_HYGIENE_PRODUCT VALUES ('SC1010', 'Deodorant', 'Fresh scent',  
'Personal Care', 3.50, 'Dove');
```

```
INSERT INTO SELF_CARE_HYGIENE_PRODUCT VALUES('N/A', 'N/A', 'N/A', 'F', 0, 'N/A');
```

--We inserted a 'N/A' tuple that represents “no products”. This is to account for instances like when the customer buys only medicines and no products, as we have some ternary relationships that merge them and use the products and medicine primary keys as the ternary relation’s composite key.

11. PRESCRIPTION:

```
INSERT INTO PRESCRIPTION VALUES (1, TO_DATE('01/11/2024', 'DD/MM/YYYY'), 2,  
TO_DATE('01/02/2025', 'DD/MM/YYYY'), '+96111234567', 'Dr. Mansour', 'DL12345', 10, 'Sami',  
'Haddad');
```

```
INSERT INTO PRESCRIPTION VALUES (2, TO_DATE('15/11/2024', 'DD/MM/YYYY'), 1,  
TO_DATE('15/12/2024', 'DD/MM/YYYY'), '+96111234568', 'Dr. Akl', 'DL54321', 15, 'Rita', 'Khalil');
```

```
INSERT INTO PRESCRIPTION VALUES (3, TO_DATE('20/11/2024', 'DD/MM/YYYY'), 3,  
TO_DATE('20/02/2025', 'DD/MM/YYYY'), '+96111234569', 'Dr. George', 'DL67890', 20, 'Maya', 'Saad');
```

```
INSERT INTO PRESCRIPTION VALUES (4, TO_DATE('22/11/2024', 'DD/MM/YYYY'), 1,  
TO_DATE('22/12/2024', 'DD/MM/YYYY'), '+96111234570', 'Dr. Samir', 'DL23456', 30, 'Ali', 'Jaber');
```

```
INSERT INTO PRESCRIPTION VALUES (5, TO_DATE('25/11/2024', 'DD/MM/YYYY'), 2,  
TO_DATE('25/02/2025', 'DD/MM/YYYY'), '+96111234571', 'Dr. Ziad', 'DL34567', 25, 'Fadi', 'Kassem');
```

```
INSERT INTO PRESCRIPTION VALUES (6, TO_DATE('30/11/2024', 'DD/MM/YYYY'), 0,  
TO_DATE('30/12/2024', 'DD/MM/YYYY'), '+96111234572', 'Dr. Akl', 'DL98765', 5, 'Samir', 'Khalil');
```

```
INSERT INTO PRESCRIPTION VALUES (7, TO_DATE('02/12/2024', 'DD/MM/YYYY'), 1,  
TO_DATE('02/01/2025', 'DD/MM/YYYY'), '+96111234573', 'Dr. Layla', 'DL67891', 18, 'Zeinab',  
'Fakhoury');
```

```
INSERT INTO PRESCRIPTION VALUES (8, TO_DATE('05/12/2024', 'DD/MM/YYYY'), 0,  
TO_DATE('05/01/2025', 'DD/MM/YYYY'), '+96111234574', 'Dr. Faris', 'DL45678', 8, 'Sami', 'Haddad');
```

```
INSERT INTO PRESCRIPTION VALUES (9, TO_DATE('10/12/2024', 'DD/MM/YYYY'), 4,  
TO_DATE('10/02/2025', 'DD/MM/YYYY'), '+96111234575', 'Dr. Rania', 'DL56789', 40, 'Layla', 'Zein');
```

```
INSERT INTO PRESCRIPTION VALUES (10, TO_DATE('12/12/2024', 'DD/MM/YYYY'), 2,  
TO_DATE('12/02/2025', 'DD/MM/YYYY'), '+96111234576', 'Dr. Hassan', 'DL65432', 20, 'Ali', 'Jaber');
```

12. Contains2:

```
INSERT INTO contains2 VALUES (1, 'A01AA01');
```

```
INSERT INTO contains2 VALUES (2, 'B02BC01');
```

```
INSERT INTO contains2 VALUES (3, 'C03CC01');
```

```
INSERT INTO contains2 VALUES (4, 'D04DC01');
```

```
INSERT INTO contains2 VALUES (5, 'E05EE01');
```

```
INSERT INTO contains2 VALUES (6, 'F06FF01');
```

```
INSERT INTO contains2 VALUES (7, 'G07GG01');
```

```
INSERT INTO contains2 VALUES (8, 'H08HH01');
```

```
INSERT INTO contains2 VALUES (9, 'I09II01');
```

```
INSERT INTO contains2 VALUES (10, 'J10JJ01');
```

13. Payment_method:

```
INSERT INTO Payment_method VALUES (1, 'Card');
INSERT INTO Payment_method VALUES (2, 'Cash');
INSERT INTO Payment_method VALUES (3, 'Card');
INSERT INTO Payment_method VALUES (4, 'Cash');
INSERT INTO Payment_method VALUES (5, 'Card');
INSERT INTO Payment_method VALUES (6, 'Cash');
INSERT INTO Payment_method VALUES (7, 'Card');
INSERT INTO Payment_method VALUES (8, 'Cash');
INSERT INTO Payment_method VALUES (9, 'Card');
INSERT INTO Payment_method VALUES (10, 'Cash');
```

14. Item_ID:

```
INSERT INTO Item_ID VALUES (1, 'SC1001');
INSERT INTO Item_ID VALUES (1, 'A01AA01');
INSERT INTO Item_ID VALUES (2, 'B02BC01');
INSERT INTO Item_ID VALUES (2, 'C03CC01');
INSERT INTO Item_ID VALUES (5, 'D04DC01');
INSERT INTO Item_ID VALUES (6, 'E05EE01');
INSERT INTO Item_ID VALUES (1, 'F06FF01');
INSERT INTO Item_ID VALUES (8, 'G07GG01');
INSERT INTO Item_ID VALUES (9, 'H08HH01');
INSERT INTO Item_ID VALUES (10, 'I09II01');
```

15. Dosage_Instructions:

INSERT INTO Dosage_Instructions VALUES (1, 'Take 1 tablet daily');

INSERT INTO Dosage_Instructions VALUES (1, 'Take 2 tablets daily');

INSERT INTO Dosage_Instructions VALUES (3, 'Take 1 tablet every 6 hours');

INSERT INTO Dosage_Instructions VALUES (4, 'Take 1 tablet twice a day');

INSERT INTO Dosage_Instructions VALUES (5, 'Apply cream to affected area twice a day');

INSERT INTO Dosage_Instructions VALUES (6, 'Take 1 tablet before meals');

INSERT INTO Dosage_Instructions VALUES (6, 'Take 2 tablets at bedtime');

INSERT INTO Dosage_Instructions VALUES (8, 'Take 1 tablet in the morning and evening');

INSERT INTO Dosage_Instructions VALUES (9, 'Take 1 tablet every 8 hours');

INSERT INTO Dosage_Instructions VALUES (10, 'Take 1 tablet as needed');

16. Supplies:

INSERT INTO Supplies VALUES ('A01AA01', '30012345678', 'SC1001', 3.50, TO_DATE('01/11/2024', 'DD/MM/YYYY'), 50);

INSERT INTO Supplies VALUES ('B02BC01', '30087654321', 'SC1002', 7.00, TO_DATE('10/11/2024', 'DD/MM/YYYY'), 100);

INSERT INTO Supplies VALUES ('C03CC01', '30011223344', 'SC1003', 5.50, TO_DATE('15/11/2024', 'DD/MM/YYYY'), 75);

INSERT INTO Supplies VALUES ('D04DC01', '30044556677', 'SC1004', 8.00, TO_DATE('20/11/2024', 'DD/MM/YYYY'), 60);

INSERT INTO Supplies VALUES ('E05EE01', '30099887766', 'SC1005', 6.00, TO_DATE('25/11/2024', 'DD/MM/YYYY'), 90);

INSERT INTO Supplies VALUES ('F06FF01', '30022334455', 'SC1006', 3.00, TO_DATE('30/11/2024', 'DD/MM/YYYY'), 120);

INSERT INTO Supplies VALUES ('G07GG01', '30055667788', 'SC1007', 4.50, TO_DATE('05/12/2024', 'DD/MM/YYYY'), 150);

```
INSERT INTO Supplies VALUES ('H08HH01', '30066778899', 'SC1008', 7.50, TO_DATE('10/12/2024', 'DD/MM/YYYY'), 200);
```

```
INSERT INTO Supplies VALUES ('I09II01', '30077889900', 'SC1009', 2.50, TO_DATE('15/12/2024', 'DD/MM/YYYY'), 180);
```

```
INSERT INTO Supplies VALUES ('J10JJ01', '30088990011', 'SC1010', 5.00, TO_DATE('20/12/2024', 'DD/MM/YYYY'), 160)
```

17. Includes:

```
INSERT INTO Includes VALUES (1, 'A01AA01', 'SC1001');
```

```
INSERT INTO Includes VALUES (2, 'B02BC01', 'SC1002');
```

```
INSERT INTO Includes VALUES (3, 'C03CC01', 'SC1003');
```

```
INSERT INTO Includes VALUES (4, 'D04DC01', 'SC1004');
```

```
INSERT INTO Includes VALUES (5, 'E05EE01', 'SC1005');
```

```
INSERT INTO Includes VALUES (6, 'F06FF01', 'SC1006');
```

```
INSERT INTO Includes VALUES (7, 'G07GG01', 'SC1007');
```

```
INSERT INTO Includes VALUES (8, 'H08HH01', 'SC1008');
```

```
INSERT INTO Includes VALUES (9, 'I09II01', 'SC1009');
```

```
INSERT INTO Includes VALUES (10, 'J10JJ01', 'SC1010');
```

18. Stores:

```
INSERT INTO Stores VALUES ('A01AA01', 'SC1001', 'I001', TO_DATE('01/10/2024', 'DD/MM/YYYY'), TO_DATE('01/10/2025', 'DD/MM/YYYY'));
```

```
INSERT INTO Stores VALUES ('B02BC01', 'SC1002', 'I002', TO_DATE('10/10/2024', 'DD/MM/YYYY'), TO_DATE('10/10/2025', 'DD/MM/YYYY'));
```

```
INSERT INTO Stores VALUES ('C03CC01', 'SC1003', 'I003', TO_DATE('15/10/2024', 'DD/MM/YYYY'), TO_DATE('15/10/2025', 'DD/MM/YYYY'));
```

```
INSERT INTO Stores VALUES ('D04DC01', 'SC1004', 'I001', TO_DATE('20/10/2024', 'DD/MM/YYYY'), TO_DATE('20/10/2025', 'DD/MM/YYYY'));
```

INSERT INTO Stores VALUES ('E05EE01', 'SC1005', 'I002', TO_DATE('25/10/2024', 'DD/MM/YYYY'), TO_DATE('25/10/2025', 'DD/MM/YYYY'));

INSERT INTO Stores VALUES ('F06FF01', 'SC1006', 'I003', TO_DATE('30/10/2024', 'DD/MM/YYYY'), TO_DATE('30/10/2025', 'DD/MM/YYYY'));

INSERT INTO Stores VALUES ('G07GG01', 'SC1007', 'I001', TO_DATE('05/11/2024', 'DD/MM/YYYY'), TO_DATE('05/11/2025', 'DD/MM/YYYY'));

INSERT INTO Stores VALUES ('H08HH01', 'SC1008', 'I002', TO_DATE('10/11/2024', 'DD/MM/YYYY'), TO_DATE('10/11/2025', 'DD/MM/YYYY'));

INSERT INTO Stores VALUES ('I09II01', 'SC1009', 'I003', TO_DATE('15/11/2024', 'DD/MM/YYYY'), TO_DATE('15/11/2025', 'DD/MM/YYYY'));

INSERT INTO Stores VALUES ('J10JJ01', 'SC1010', 'I001', TO_DATE('20/11/2024', 'DD/MM/YYYY'), TO_DATE('20/11/2025', 'DD/MM/YYYY'));

TABLE STATES:

1. EMPLOYEE

SELECT * FROM EMPLOYEE;

SSN	FNAME	LNAME	DOB	SEX	POSITION	LICENSE_NUMBER	SALARY	PHONE_NUMBER	EMAIL	CITY	STREET_NB	STREET_NAME	APARTMENT_NUMBER	B_ID	START_DATE	END_DATE	HOURS_WORKED
112233445	Fadi	Kassem	08/23/1980	M	Accountant	-	2500	+96171234567	fadi.kassem@healerdealer.com	Beirut	10	Achrafieh St	3	B003	02/01/2023	-	45
223344556	Layla	Zein	12/16/1987	F	Pharmacist	PH654321	1800	+96174567890	layla.zein@healerdealer.com	Beirut	50	Karatina St	4	B001	03/05/2022	-	40
334455667	Hassan	Amin	09/25/1992	M	Manager	-	3000	+96180567890	hassan.amin@healerdealer.com	Beirut	10	Verdan St	5	B002	07/01/2022	-	50
445566778	Maya	Saad	03/14/1991	F	Cashier	-	1100	+96170654321	maya.saad@healerdealer.com	Beirut	75	Sin El Fil St	6	B003	08/11/2022	-	30
556677889	Ali	Jaber	05/03/1988	M	Cleaner	-	800	+96195543210	ali.jaber@healerdealer.com	Beirut	1	Hamra St	7	B001	09/20/2022	-	40
667788990	Rita	Daher	07/20/1989	F	Pharmacist	PH789123	2200	+96191567890	rita.daher@healerdealer.com	Beirut	10	Achrafieh St	8	B002	06/12/2022	-	42
778899001	Samir	Khalil	11/09/1993	M	Manager	-	1000	+96173567890	samir.khalil@healerdealer.com	Beirut	75	Sin El Fil St	9	B003	05/19/2022	-	38
889900112	Zeinab	Fakhoury	01/02/1986	F	Accountant	-	2700	+96182567890	zeinab.fakhoury@healerdealer.com	Beirut	30	Monot St	10	B001	08/09/2022	-	45
123456789	Rami	Haddad	02/12/1985	M	Manager	-	2000	+96112345678	rami.haddad@healerdealer.com	Beirut	1	Hamra St	3	B001	01/01/2022	-	40
987654321	Noura	Elia	06/05/1990	F	Cashier	-	1200	+96176345678	noura.elia@healerdealer.com	Beirut	5	Verdan St	5	B002	07/15/2022	-	35

2. BRANCH

SELECT * FROM BRANCH;

BRANCHID	NAME	PHONE	EMAIL	STREET	CITY	MANAGER_NAME	MANAGER_SSN
B003	Achrafieh HD Branch	+96117123456	achrafieh@healerdealer.com	Achrafieh St	Beirut	Samir Khalil	778899001
B001	Hamra HD Branch	+96119876543	hamra@healerdealer.com	Hamra Street	Beirut	Rami Haddad	123456789
B002	Bliss HD Branch	+96119987654	bliss@healerdealer.com	Bliss St	Beirut	Hassan Amin	334455667

3. SUPPLIER

SELECT * FROM SUPPLIER;

SUPPLIERID	NAME	CONTACT_PERSON	DELIVERY_TERM	EMAIL	PHONE_NUMBER	STREET	CITY
30012345678	Beirut Pharma Solutions	Joseph Atallah	Monthly	contact@pharmaleb.com	+96130123456	Jal el-Dib Street	Metn
30087654321	Cedars Medical Supply Co.	Samir Bassil	Weekly	info@medico.com	+96130876543	Zalka Main Street	Metn
30011223344	LebHealth Pharmaceuticals	Rania Daher	Yearly	sales@healthplus.com	+96130456789	Chiah Street	Beirut
30044556677	Mount Lebanon Drug Distributors	Kassem Faris	Weekly	medleben@med.com	+96131234567	Maameltein Street	Jounieh
30099887766	Phoenicia Medica	Yasmin Akram	Monthly	biopharma@bph.com	+96133234567	Riad el-Solh Street	Beirut
30022334455	MedCare Solutions Lebanon	Khaled Taha	Yearly	info@lebmed.com	+96135098765	Beirut Souks Street	Beirut
30055667788	Cedar Valley Pharma	Maya Shamas	Monthly	contact@careplus.com	+96134356789	Hamra Street	Beirut
30066778899	Lebanese Life Sciences Supply	Rami Saba	Weekly	info@healthcareco.com	+96136457890	City Centre	Beirut
30077889900	Sahha Pharma Distributors	Zahra Harb	Weekly	sales@medicare.com	+96137548901	Ain Mreisseh Street	Beirut
30088990011	AlphaMed Lebanon	Jad Khoury	Monthly	contact@pharmamed.com	+96138765012	Rue de Verdun	Beirut

4. INVENTORY

SELECT * FROM INVENTORY;

INVENTORYID	QUANTITY	STREET	CITY	B_ID
I001	100	Hamra St	Beirut	B001
I002	200	Bliss St	Beirut	B002
I003	150	Achrafieh St	Beirut	B003

5. CUSTOMER

SELECT * FROM CUSTOMER;

FNAME	LNAME	BILL_ID	PHONE_NUMBER	DOB
Maya	Saad	3	+96133333333	02/08/1990
Ali	Jaber	4	+96144444444	03/14/1987
Zeinab	Fakhoury	7	+96155555555	04/20/1992
Fadi	Kassem	5	+96166666666	07/12/1980
Samir	Khalil	6	+96177777777	09/22/1989
Hassan	Amin	9	+96188888888	06/05/1993
Layla	Zein	8	+96199999999	08/13/1986
Rami	Haddad	10	+96110000000	11/17/1984
Sami	Haddad	1	+96111111111	05/05/1995
Rita	Khalil	2	+96122222222	10/10/1985

6. BILLING_INVOICE

SELECT * FROM BILLING_INVOICE;

INVOICEID	ESSN	DATE_ISSUED	TOTAL_AMOUNT	PAYMENT_STATUS	CF_NAME	CL_NAME	AMOUNT_PAID
3	112233445	11/19/2024	30	paid	Maya	Saad	30
4	223344556	11/21/2024	40	pending	Ali	Jaber	10
5	334455667	11/23/2024	50	paid	Fadi	Kassem	50
6	445566778	11/25/2024	60	pending	Samir	Khalil	30
7	556677889	11/26/2024	70	paid	Zeinab	Fakhoury	70
8	667788990	11/28/2024	80	pending	Layla	Zein	40
9	778899001	11/30/2024	90	paid	Hassan	Amin	90
10	889900112	12/01/2024	100	pending	Rami	Haddad	50
1	123456789	11/02/2024	15	paid	Sami	Haddad	15
2	987654321	11/16/2024	20	pending	Rita	Khalil	10

7. DEPENDENT

SELECT * FROM DEPENDENT;

FNAME	LNAME	ESSN	SEX	DOB	RELATIONSHIP
Lila	Haddad	123456789	F	10/10/2012	Child
Nadim	Haddad	123456789	M	05/15/2010	Child
Maya	Amin	334455667	F	06/22/2015	Spouse
Samir	Saad	445566778	M	09/30/2012	Child
Tariq	Jaber	556677889	M	02/02/2014	Child
Rania	Zein	667788990	F	11/10/2013	Spouse
Layla	Saad	778899001	F	04/14/2013	Child
Fadi	Kassem	889900112	M	03/25/2016	Child
Zeinab	Fakhoury	889900112	F	07/28/2011	Child
Rami	Haddad	123456789	M	05/02/2009	Spouse

8. PAYMENT_TERM

SELECT * FROM Payment_term;

SUPPLIERID	PAYMENT_TERM
30011223344	Yearly
30012345678	Monthly
30022334455	Yearly
30044556677	Monthly
30055667788	Monthly
30066778899	Weekly
30077889900	Monthly
30087654321	Weekly
30088990011	Yearly
30099887766	Weekly

9. MEDICINE

SELECT * FROM MEDICINE;

MEDICINEID	NAME	DESCRIPTION	PRESCRIPTION_REQUIRED	PRICE	MANUFACTURER
A01AA01	Paracetamol	Pain reliever	T	5	PharmaLeb
B02BC01	Ibuprofen	Anti-inflammatory	T	10	Medico
C03CC01	Aspirin	Blood thinner	T	7	HealthPlus
D04DC01	Loratadine	Antihistamine	T	8.5	PharmaLeb
E05EE01	Cetirizine	Antihistamine	T	6.5	Medico
F06FF01	Hydrocortisone	Steroid	T	12	HealthPlus
G07GG01	Ciprofloxacin	Antibiotic	T	15	PharmaLeb
H08HH01	Clindamycin	Antibiotic	T	13	Medico
I09II01	Amoxicillin	Antibiotic	T	9	HealthPlus
J10JJ01	Omeprazole	Acid reducer	T	10.5	PharmaLeb
N/A	N/A	N/A	F	0	N/A

10. SELF_CARE_HYGIENE_PRODUCT

SELECT * FROM SELF_CARE_HYGIENE_PRODUCT;

PRODUCTID	NAME	DESCRIPTION	CATEGORY	PRICE	MANUFACTURER
SC1001	Toothpaste	Mint flavor	Dental	3	OralB
SC1002	Shampoo	For dry hair	Hair Care	8	Dove
SC1003	Body Lotion	Moisturizing	Skin Care	5.5	Nivea
SC1004	Face Wash	For sensitive skin	Skin Care	6	Neutrogena
SC1005	Conditioner	For curly hair	Hair Care	7	Pantene
SC1006	Hand Soap	Lavender scent	Personal Care	2.5	Lifebuoy
SC1007	Shaving Cream	Sensitive skin	Personal Care	4	Gillette
SC1008	Shower Gel	For men	Skin Care	6.5	Old Spice
SC1009	Lip Balm	Nourishing	Personal Care	2	ChapStick
SC1010	Deodorant	Fresh scent	Personal Care	3.5	Dove
N/A	N/A	N/A	F	0	N/A

11. PRESCRIPTION

SELECT * FROM PRESCRIPTION;

PRESCRIPTIONID	DATE_ISSUED	REFILL_COUNT	EXPIRY	DR_PHONE_NB	DR_NAME	DR_LICENSE_NUMBER	REFILLS_FREQUENCY	CF_NAME	CL_NAME
1	11/01/2024	2	02/01/2025	+96111234567	Dr. Mansour	DL12345	10	Sami	Haddad
2	11/15/2024	1	12/15/2024	+96111234568	Dr. Akl	DL54321	15	Rita	Khalil
3	11/20/2024	3	02/20/2025	+96111234569	Dr. George	DL67890	20	Maya	Saad
4	11/22/2024	1	12/22/2024	+96111234570	Dr. Samir	DL23456	30	Ali	Jaber
5	11/25/2024	2	02/25/2025	+96111234571	Dr. Ziad	DL34567	25	Fadi	Kassem
6	11/30/2024	0	12/30/2024	+96111234572	Dr. Akl	DL98765	5	Samir	Khalil
7	12/02/2024	1	01/02/2025	+96111234573	Dr. Layla	DL67891	18	Zeinab	Fakhoury
8	12/05/2024	0	01/05/2025	+96111234574	Dr. Faris	DL45678	8	Sami	Haddad
9	12/10/2024	4	02/10/2025	+96111234575	Dr. Rania	DL56789	40	Layla	Zein
10	12/12/2024	2	02/12/2025	+96111234576	Dr. Hassan	DL65432	20	Ali	Jaber

12. CONTAINS2

SELECT * FROM contains2;

PID	MID
1	A01AA01
2	B02BC01
3	C03CC01
4	D04DC01
5	E05EE01
6	F06FF01
7	G07GG01
8	H08HH01
9	I09II01
10	J10JJ01

13. PAYMENT_METHOD

SELECT * FROM Payment_method;

INVOICEID	PAYMENT_METHOD
1	Card
2	Cash
3	Card
4	Cash
5	Card
6	Cash
7	Card
8	Cash
9	Card
10	Cash

14. ITEM_ID

SELECT * FROM Item_ID

INVOICEID	ITEMID
1	A01AA01
1	F06FF01
1	SC1001
2	B02BC01
2	C03CC01
5	D04DC01
6	E05EE01
8	G07GG01
9	H08HH01
10	I09II01

15- DOSAGE_INSTRUCTIONS

SELECT * FROM Dosage_Instructions;

PRESCRIPTIONID	DOSAGE_INSTRUCTION
1	Take 1 tablet daily
1	Take 2 tablets daily
3	Take 1 tablet every 6 hours
4	Take 1 tablet twice a day
5	Apply cream to affected area twice a day
6	Take 1 tablet before meals
6	Take 2 tablets at bedtime
8	Take 1 tablet in the morning and evening
9	Take 1 tablet every 8 hours
10	Take 1 tablet as needed

16- SUPPLIES

SELECT * FROM Supplies;

MID	SID	PRID	PRICE_PER_UNIT	SUPPLY_DATE	QUANTITY_SUPPLIED
C03CC01	30011223344	SC1003	5.5	11/15/2024	75
D04DC01	30044556677	SC1004	8	11/20/2024	60
E05EE01	30099887766	SC1005	6	11/25/2024	90
A01AA01	30012345678	SC1001	3.5	11/01/2024	50
B02BC01	30087654321	SC1002	7	11/10/2024	100
F06FF01	30022334455	SC1006	3	11/30/2024	120
G07GG01	30055667788	SC1007	4.5	12/05/2024	150
H08HH01	30066778899	SC1008	7.5	12/10/2024	200
I09II01	30077889900	SC1009	2.5	12/15/2024	180
J10JJ01	30088990011	SC1010	5	12/20/2024	160

17- INCLUDES

SELECT * FROM Includes;

BID	MID	PRID
1	A01AA01	SC1001
2	B02BC01	SC1002
3	C03CC01	SC1003
4	D04DC01	SC1004
5	E05EE01	SC1005
6	F06FF01	SC1006
7	G07GG01	SC1007
8	H08HH01	SC1008
9	I09II01	SC1009
10	J10JJ01	SC1010

18- STORES

SELECT * FROM Stores;

MID	PRID	INID	RESTOCK_DATE	EXPIRY
A01AA01	SC1001	I001	10/01/2024	10/01/2025
B02BC01	SC1002	I002	10/10/2024	10/10/2025
C03CC01	SC1003	I003	10/15/2024	10/15/2025
D04DC01	SC1004	I001	10/20/2024	10/20/2025
E05EE01	SC1005	I002	10/25/2024	10/25/2025
F06FF01	SC1006	I003	10/30/2024	10/30/2025
G07GG01	SC1007	I001	11/05/2024	11/05/2025
H08HH01	SC1008	I002	11/10/2024	11/10/2025
I09II01	SC1009	I003	11/15/2024	11/15/2025
J10JJ01	SC1010	I001	11/20/2024	11/20/2025

Transactions

Query 1: Identifying and Removing Expired Medicines

Problem:

The pharmacy needs to ensure its inventory is free of expired medicines to avoid legal liabilities and ensure customer safety. The procedure for handling expired medications in a pharmacy inventory system is covered by this query.

However, since we haven't had any expired medications in our database yet, we must first insert a few tuples in the INVENTORY table that include expired medicine. This will help us test out this transaction and prove the ability of our database to perform such an important function.

So, the procedures entail adding, recognizing, alerting, and expunging expired medications from the database.

Query:

- 1- Adding expired medicine values to the INVENTORY table:

```
INSERT INTO Stores VALUES ('E05EE01', 'N/A', 'I002', TO_DATE('01/10/2022', 'DD/MM/YYYY'),  
TO_DATE('01/10/2023', 'DD/MM/YYYY'));
```

```
INSERT INTO Stores VALUES ('A01AA01', 'N/A', 'I001', TO_DATE('01/10/2020', 'DD/MM/YYYY'),  
TO_DATE('01/10/2023', 'DD/MM/YYYY'));
```

- 2- Identify expired medicines in inventory.

```
SELECT  
  M.MedicineID, M.Name AS Medicine_Name, S.Expiry, I.InventoryID,  
  (SELECT B.Name FROM BRANCH B WHERE B.BranchID = I.B_ID) AS Branch_Name  
FROM  
  MEDICINE M, STORES S, INVENTORY I  
WHERE  
  M.MedicineID = S.mID  
  AND S.inID = I.InventoryID  
  AND S.Expiry < SYSDATE;
```

MEDICINEID	MEDICINE_NAME	EXPIRY	INVENTORYID	BRANCH_NAME
E05EE01	Cetirizine	10/01/2023	I002	Bliss HD Branch
A01AA01	Paracetamol	10/01/2023	I001	Hamra HD Branch

3- Identify suppliers of expired medicines to notify them of restocking

```

SELECT
    Su.Name AS Supplier_Name, Su.Email AS Supplier_Email,
    M.MedicineID, M.Name AS Medicine_Name
FROM
    SUPPLIER Su, MEDICINE M, Supplies S
WHERE
    Su.SupplierID = S.SID
AND S.MID= M.MedicineID
AND M.MedicineID IN (
    SELECT M.MedicineID
    FROM MEDICINE M, STORES S
    WHERE M.MedicineID = S.mID AND S.Expiry < SYSDATE
);

```

SUPPLIER_NAME	SUPPLIER_EMAIL	MEDICINEID	MEDICINE_NAME
Beirut Pharma Solutions	contact@pharmaleb.com	A01AA01	Paracetamol
Phoenicia Medica	biopharma@bph.com	E05EE01	Cetirizine

4- Remove expired medicines from inventory

```

DELETE FROM STORES S
WHERE S.mID IN (
    SELECT M.MedicineID FROM MEDICINE M WHERE M.MedicineID = S.mID AND S.Expiry <
    SYSDATE );

```

Before:

MID	PRID	INID	RESTOCK_DATE	EXPIRY
B02BC01	SC1002	I002	10/10/2024	10/10/2025
C03CC01	SC1003	I003	10/15/2024	10/15/2025
D04DC01	SC1004	I001	10/20/2024	10/20/2025
F06FF01	SC1006	I003	10/30/2024	10/30/2025
G07GG01	SC1007	I001	11/05/2024	11/05/2025
H08HH01	SC1008	I002	11/10/2024	11/10/2025
I09II01	SC1009	I003	11/15/2024	11/15/2025
J10JJ01	SC1010	I001	11/20/2024	11/20/2025
A01AA01	SC1001	I001	10/01/2024	10/01/2025
E05EE01	SC1005	I002	10/25/2024	10/25/2025
A01AA01	N/A	I001	10/01/2020	10/01/2023
E05EE01	N/A	I002	10/01/2022	10/01/2023

12 rows returned in 0.00 seconds [Download](#)

After:

MID	PRID	INID	RESTOCK_DATE	EXPIRY
B02BC01	SC1002	I002	10/10/2024	10/10/2025
C03CC01	SC1003	I003	10/15/2024	10/15/2025
D04DC01	SC1004	I001	10/20/2024	10/20/2025
F06FF01	SC1006	I003	10/30/2024	10/30/2025
G07GG01	SC1007	I001	11/05/2024	11/05/2025
H08HH01	SC1008	I002	11/10/2024	11/10/2025
I09II01	SC1009	I003	11/15/2024	11/15/2025
J10JJ01	SC1010	I001	11/20/2024	11/20/2025
A01AA01	SC1001	I001	10/01/2024	10/01/2025
E05EE01	SC1005	I002	10/25/2024	10/25/2025

10 rows returned in 0.00 seconds [Download](#)

Explanation:

Filling the inventory with sample medication information, including expiration dates, is the initial step. Records with fields like **MedicineID**, **InventoryID**, and **expiration dates** are added to the Stores table using the INSERT INTO commands. For instance, a medication with ID E05EE01 that expired on January 10, 2023, is inserted in the first record.

Using a SELECT query, the second move finds medications that have passed their expiration date. The query retrieves information about the medication, including its name, expiration date, MedicineID, and branch name. To make sure that only expired medications are retrieved, it employs conditions such as

S.Expiry < SYSDATE. The **BRANCH** table is used in a subquery within the SELECT clause to link the inventory branch name.

In order for suppliers to resupply, the third stage makes sure they are notified when medications expire. In addition to the **MedicineID** and **Medicine_Name** for expired medications, the query receives the supplier's information (Name, Email). It filters medications whose expiration dates are past SYSDATE using a subquery. To help with restocking decisions, a subquery also determines the overall amount of expired medication delivered.

Using a DELETE query, the last step eliminates expired medications from the inventory. When the drug ID matches entries with expiration dates earlier than the current system date (SYSDATE), the query removes those records from the **Stores** table. These expired medications are located using a subquery in the WHERE clause.

Query 2: A new customer purchases items.

Problem:

A new customer named "Evelyn Harper" comes to purchase an item. We have to add our customer first in the CUSTOMER TABLE using the same way we did in the insertion phase. We insert a new tuple for the billing invoice and payment method for her purchase while updating includes table and items_ID.

Query:

1- Insert customer and their bill:

```
ALTER TABLE CUSTOMER DROP CONSTRAINT PK_CUSTOMER ;  
--we previously changed the name of the constraint.
```

```
ALTER TABLE CUSTOMER MODIFY Bill_ID INT NULL;
```

```
INSERT INTO CUSTOMER VALUES ('Evelyn', 'Harper', NULL, '+96111100000',  
TO_DATE('05/05/2000', 'DD/MM/YYYY'));
```

```
INSERT INTO BILLING_INVOICE VALUES (22, '123456789', TO_DATE('06/11/2024',
'DD/MM/YYYY'), 20.00, 'paid', 'Evelyn', 'Harper', 15.00);
```

```
UPDATE CUSTOMER
```

```
SET Bill_ID = 22
```

```
WHERE Fname = 'Evelyn' AND Lname ='Harper';
```

```
ALTER TABLE CUSTOMER ADD CONSTRAINT PK_CUSTOMER PRIMARY KEY (Fname, Lname,
Bill_ID);
```

2- Insert items bought and payment methods used in the purchase.

```
INSERT INTO Item_ID VALUES (22, 'SC1001');
```

```
INSERT INTO Payment_method VALUES (22, 'Cash');
```

```
INSERT INTO Includes Values(22,'N/A','SC1001');
```

Output:

CUSTOMER:

FNAME	LNAME	BILL_ID	PHONE_NUMBER	DOB
Evelyn	Harper	22	+96111100000	05/05/2000

BILLING_INVOICE:

INVOICEID	ESSN	DATE_ISSUED	TOTAL_AMOUNT	PAYMENT_STATUS	CF_NAME	CL_NAME	AMOUNT_PAID
22	123456789	11/06/2024	20	paid	Evelyn	Harper	15

Item_ID:

INVOICEID	ITEMID
22	SC1001

Includes:

BID	MID	PRID
22	N/A	SC1001

Payment method:

INVOICEID	PAYMENT_METHOD
22	Cash

Query 3: A new customer comes with a prescription

Problem:

Another customer named "Omar Farouq" comes in with a prescription. We have to add our customer first in the CUSTOMER TABLE using the same way we did in the insertion phase. Then we simply add the prescription while updating the contains2 table to indicate what medicine was included. We also insert a new tuple for the billing invoice and payment method for his purchase, while updating includes table and items_ID.

Query:

1- Insert customer and their bill:

```
ALTER TABLE CUSTOMER DROP CONSTRAINT PK_CUSTOMER ;
```

```
INSERT INTO CUSTOMER VALUES ('Omar', 'Farouq', NULL, '+96170250430',  
TO_DATE('05/05/2000', 'DD/MM/YYYY'));
```

```
INSERT INTO BILLING_INVOICE VALUES (23, '123456789', TO_DATE('08/11/2024',  
'DD/MM/YYYY'), 9.00, 'paid', 'Omar', 'Farouq', 9.00);
```

```
UPDATE CUSTOMER  
SET Bill_ID = 23  
WHERE Fname = 'Omar' AND Lname ='Farouq';
```

```
ALTER TABLE CUSTOMER ADD CONSTRAINT PK_CUSTOMER PRIMARY KEY (Fname, Lname,  
Bill_ID);
```

2- Add a new PRESCRIPTION for the customer:

```
INSERT INTO PRESCRIPTION VALUES (22, TO_DATE('12/12/2024', 'DD/MM/YYYY'), 2,  
TO_DATE('12/02/2025', 'DD/MM/YYYY'), '+96111234576', 'Dr. Hassan', 'DL65432', 30, 'Omar',  
'Farouq');
```

3- Add medicines to the prescription in CONTAINS2

```
INSERT INTO contains2 VALUES (22, 'I09II01');
```

4- Add the related items in ITEM_ID and add the payment method:

```
INSERT INTO Item_ID VALUES (23, 'I09II01');  
INSERT INTO Payment_method VALUES (23, 'Cash');  
INSERT INTO includes VALUES (23, 'I09II01', 'N/A');
```

Output:

CUSTOMER:

FNAME	LNAME	BILL_ID	PHONE_NUMBER	DOB
Omar	Farouq	23	+96170250430	05/05/2000

BILLING_INVOICE:

INVOICEID	ESSN	DATE_ISSUED	TOTAL_AMOUNT	PAYMENT_STATUS	CF_NAME	CL_NAME	AMOUNT_PAID
23	123456789	11/08/2024	9	paid	Omar	Farouq	9

PRESCRIPTION:

PRESCRIPTIONID	DATE_ISSUED	REFILL_COUNT	EXPIRY	DR_PHONE_NB	DR_NAME	DR_LICENSE_NUMBER	REFILLS_FREQUENCY	CF_NAME	CL_NAME
22	12/12/2024	2	02/12/2025	+96111234576	Dr. Hassan	DL65432	30	Omar	Farouq

Contains2:

PID	MID
22	I09II01

Item_ID:

INVOICEID	ITEMID
23	I09II01

Includes:

BID	MID	PRID
23	I09II01	N/A

Payment_method:

INVOICEID	PAYMENT_METHOD
23	Cash

Query 4: List All Employees and Their Assigned Branches

Problem:

The pharmacy needs to track employee assignments to branches for effective staff management and smooth operations. This query retrieves each employee's first and last name, along with their assigned departments by matching the B_ID column in the EMPLOYEE table to the BranchID column in the Branch table.

This clear presentation of staffing allows managers to quickly see which employees are at each branch, enhancing operational efficiency, staff scheduling, and resource allocation. For instance, this information helps identify branches that may be understaffed and ensures transparency in staff deployments across various locations.

Query:

```
SELECT
    E.Fname,
    E.Lname,
    B.Name AS Branch_Name
FROM
    EMPLOYEE E,
    Branch B
WHERE
    E.B_id = B.BranchID;
```

Output

FNAME	LNAME	BRANCH_NAME
Fadi	Kassem	Achrafieh HD Branch
Layla	Zein	Hamra HD Branch
Hassan	Amin	Bliss HD Branch
Maya	Saad	Achrafieh HD Branch
Ali	Jaber	Hamra HD Branch
Rita	Daher	Bliss HD Branch
Samir	Khalil	Achrafieh HD Branch
Zeinab	Fakhoury	Hamra HD Branch
Rami	Haddad	Hamra HD Branch
Noura	Elia	Bliss HD Branch
Hamad	Awada	Zalka Branch

Query 5: Calculate Total Sales for Each Medicine

Problem:

The query provides an approximation of sales for each drug type. The total sales for each medicine is calculated by counting how many times each medicine appears in the INCLUDES table (using COUNT(*)), then multiplying that count by the price of the medicine (M.Price).

We can then display the medicine name with its total sales, which will help our pharmacy know which medications are in high demand and require larger quantities to be supplied.

Query:

```
SELECT
  M.Name,
  (SELECT COUNT(*) * M.Price
   FROM INCLUDES I
   WHERE I.mID = M.MedicineID) AS Total_Sales
FROM
  MEDICINE M;
```

Output:

NAME	TOTAL_SALES
Paracetamol	5
Ibuprofen	10
Aspirin	7
Loratadine	8.5
Cetirizine	6.5
Hydrocortisone	12
Ciprofloxacin	15
Clindamycin	13
Amoxicillin	27
Omeprazole	10.5

Query 6: Add a New Branch and Hiring a Manager

Problem:

On the 1st of January 2024, HealerDealer Pharmacy decided to expand its operations by opening a new branch in Zalka. This new branch aims to cater to a larger customer base and improve the availability of services in the Mount Lebanon area. However, this expansion requires careful planning and updates to the database to ensure the smooth addition of the branch. The task involves recording all necessary details of the new branch, including its name, location, contact information, and email address, while assigning a manager to oversee its operations. The manager will be responsible for ensuring the branch's integration into the pharmacy's network and overseeing day-to-day management tasks. Without accurately recording these details, the pharmacy risks facing inefficiencies, management issues, or operational disruptions.

To address this issue, the director of HealerDealer Pharmacy tasked the database administrators with ensuring the seamless inclusion of the branch and manager into the system. This required the addition of the branch details to the database, the hiring of a qualified manager, and updating the database to reflect the linkage between the manager and the branch.

The database team implemented a three-step solution to address the problem. First, they hired and recorded the details of a new manager in the EMPLOYEE table. The manager, Hamad Awada, was hired with a monthly salary of 7,000 and other relevant details, such as their SSN, phone number, email, address, and hiring date, were stored. At this stage, the B_id field (branch ID) was left as NULL, as the branch itself had not yet been created. The second step involved adding the new branch, called Zalka Branch, into the Branch table with its name, contact information, location, and the manager's name and SSN. Finally, the EMPLOYEE table was updated to associate the manager with the new branch by setting their B_id to the branch ID of Zalka Branch, ensuring consistency across the database.

This process begins with the addition of the manager's details to the EMPLOYEE table, ensuring their personal and professional information is securely recorded in the system. Recording the manager's SSN, position, salary, and other key details lays the groundwork for their eventual assignment to a branch. The next step involves adding the branch details into the Branch table, which includes the branch's name,

contact information, location, and the assigned manager's details. This step ensures the new branch is properly registered and linked to its manager in the system. Finally, the third query updates the manager's B_id field in the EMPLOYEE table to reflect their association with the new branch. By linking the manager's record to the branch, the database creates a clear and traceable relationship between the two, ensuring operational efficiency.

Query:

1- First, we hire a new employee to be a manager and include in the EMPLOYEE table:

```
INSERT INTO EMPLOYEE VALUES ('660099009', 'Hamad', 'Awada', TO_DATE('05/03/1980',
'DD/MM/YYYY'), 'M', 'Manager', NULL, 7000.00, '+96180456890', 'johndoe@healerdealer.com',
'Bakaata',00, 'Souq Bakaata St', 101, NULL, TO_DATE('01/01/2024', 'DD/MM/YYYY'), NULL, 0);
```

2- Add a new branch and assign a Manager

```
INSERT INTO Branch VALUES ('B004', 'Zalka Branch', '+96155667788', 'zalka@healerdealer.com',
'Zalka Hwy', 'Mount Lebanon', 'Hamad Awada', '660099009');
```

3- Update Employee Information to assign the new Manager to the new branch. (updating B_ID)

```
UPDATE EMPLOYEE
```

```
SET B_id= 'B004'
```

```
WHERE SSN= '660099009'
```

--After the branch is created, this query updates the B_id field in the EMPLOYEE table to assign the manager (Hamad Awada) to the new branch (B004).

Output:

SSN	FNAME	LNAME	DOB	SEX	POSITION	LICENSE_NUMBER	SALARY	PHONE_NUMBER	EMAIL
660099009	Hamad	Awada	03/05/1980	M	Manager	-	7000	+96180456890	johndoe@healerdealer.com

CITY	STREET_NB	STREET_NAME	APARTMENT_NUMBER	B_ID	START_DATE	END_DATE	HOURS_WORKED
Bakaata	0	Souq Bakaata St	101	B004	01/01/2024	-	0

Query 7: Dosage Instructions for a Prescription

Problem:

Patients often require detailed and accurate dosage instructions to use their prescribed medications safely. Without clear and accessible records, there is a risk of confusion, misuse, or even harm. In this case, the pharmacy needs to ensure that dosage instructions are recorded accurately in the database and linked to the corresponding prescription. Additionally, pharmacists need to retrieve specific dosage instructions for a customer named Ali Jaber to provide personalized service.

Proper linkage between dosage instructions, prescriptions, and customer details is critical for operational efficiency, patient safety, and trust in the pharmacy's services.

To address this issue, we take two steps. First, we record dosage instructions for a specific prescription in the database. Second, we retrieve the dosage instructions for Ali Jaber, including the medication name, dosage information, and customer details. The SQL queries required for these tasks ensure that the database efficiently handles the data while maintaining accuracy.

Query:

Part 1: Recording Dosage Instructions

The first query records dosage instructions for a prescription with ID 1. These instructions specify that the medication should be taken as "1 tablet every 6 hours for 3 days." This ensures that the pharmacy can provide patients with consistent and reliable guidelines for using their prescribed medication.

```
INSERT INTO Dosage_Instructions (PrescriptionID, Dosage_instruction)
VALUES (1, 'Take 1 tablet every 6 hours for 3 days');
```

Part 2: Retrieving Dosage Instructions for Ali Jaber

The second query retrieves detailed information for Ali Jaber, including their first and last name, the name of the prescribed medication, and the dosage instructions linked to the prescription. The query uses joins across multiple tables (CUSTOMER, PRESCRIPTION, Dosage_Instructions, MEDICINE, and CONTAINS2) to ensure that all relevant data is fetched accurately.

```
SELECT
    C.Fname,
    C.Lname,
    M.Name,
    D.Dosage_Instruction
```

```

FROM
    Dosage_Instructions D,
    CUSTOMER C,
    MEDICINE M,
    PRESCRIPTION P,
    CONTAINS2 CT
WHERE
    P.CF_name = C.Fname
    AND P.CL_name = C.Lname
    AND P.PrescriptionID = D.PrescriptionID
    AND CT.PID = P.PrescriptionID
    AND CT.mid = M.MedicineID
    AND C.Lname = 'Jaber'
    AND C.Fname = 'Ali';

```

QUERY 8: Medication Recall Management

Problem:

A pharmaceutical company has issued a recall for a batch of medication with ID 'G07GG01' due to contamination concerns. As such, our pharmacy was notified of this alert because we recently bought a batch from a supplier on 05/12/2024.

Us database administrators were responsible for finding all customers who purchased this contaminated medicine of MedicineID 'G07GG01' and getting their numbers to initiate its return or replacement.

So this query will search for all **customers** with **billing invoices** that **include** the **medicine** of ID 'G07GG01' **supplied** on 05/12/2024. So of course our query will contain the related tables of CUSTOMER, BILLING_INVOICE, Includes, MEDICINE, and supplies.

Query:

```

SELECT
    C.Fname,
    C.Lname,
    C.Phone_number,

```

```

M.Name AS Medicine,
B.Date_issued
FROM
CUSTOMER C,
BILLING_INVOICE B,
INCLUDES I,
MEDICINE M,
SUPPLIES S
WHERE
C.Bill_ID = B.InvoiceID
AND B.InvoiceID = I.bID
AND I.mID = M.MedicineID
AND M.MedicineID = S.mID
AND M.MedicineID = 'G07GG01'
AND S.supply_date IN TO_DATE('05/12/2024', 'DD/MM/YYYY');

```

Output: The output will display the customers' full names, contact information, medicine name, and dates of purchase, allowing employees to initiate the medicine's replacement or recall.

FNAME	LNAME	PHONE_NUMBER	MEDICINE	DATE_ISSUED
Zeinab	Fakhoury	+96155555555	Ciprofloxacin	11/26/2024

QUERY 9: Supplier Payment Summary for Auditing

Problem:

The pharmacy needs to audit all payments made to suppliers over the past quarter for compliance and reconciliation.

Query:

```
SELECT

    S.Name AS Supplier,

    P.Payment_term,

    SUM(SU.Quantity_supplied * SU.Price_per_unit) AS Total_Payment

FROM

    SUPPLIER S,

    SUPPLIES SU,

    Payment_term P

WHERE

    S.SupplierID = SU.sID

    AND S.SupplierID = P.SupplierID

    AND SU.Supply_date BETWEEN TO_DATE('01/11/2024', 'DD/MM/YYYY') AND
    TO_DATE('20/12/2024', 'DD/MM/YYYY')

GROUP BY

    S.SupplierID, S.Name, P.Payment_term;
```

Explanation:

To guarantee compliance and make reconciliation easier, the pharmacy must audit all supplier payments made during the previous quarter. The query groups the data by supplier and payment terms and determines the total amount paid to each provider throughout the given time period. Each **supplier's name, terms of payment, and total amount paid** are retrieved by the query. For each supply record, it multiplies the quantity of products supplied by the price per unit to determine the total payment. Using their respective identities, the query connects the **SUPPLIER**, **SUPPLIES**, and **Payment_term** tables. Only records that fall inside the designated date range (November 1, 2024, to December 20, 2024) are taken into consideration thanks to the WHERE clause.

To determine the total payment amount for each supplier, the GROUP BY clause arranges the data according to the supplier's name, ID, and terms of payment.

- SELECT P.Payment_term, SUM, S.Name AS Supplier(SU.Price_per_unit * SU.Quantity_supplied) AS Total Amount Paid: The supplier name and terms of payment are

chosen in this section, and the total amount due is determined by multiplying the quantity delivered by the unit price and adding the results. FROM SUPPLIER S, SUPPLIES SU, Payment_term P: Payment terms, supply transactions, and supplier information are retrieved from these tables.

- WHERE P.SupplierID AND SU.Supply_date plus S.SupplierID = SU.sID AND S.SupplierID IN CONTRAST WITH TO_DATE('01/11/2024', 'DD/MM/YYYY') AND TO_DATE('20/12/2024', 'DD/MM/YYYY'): This limits the results to supplies made between November 1, 2024, and December 20, 2024, joins the tables via supplier IDs, and filters the data to include only supplies linked with the designated suppliers.
- GROUP BY S. Name, S. Supplier ID, and P. Payment Term: This ensures that the total payment is determined for each supplier separately by grouping the results by supplier ID, supplier name, and payment terms. The pharmacy can confirm compliance and precisely reconcile payments made within the designated time frame thanks to this query, which guarantees a thorough audit of supplier payments.

Output:

A summary report of total payments made to each supplier during the specified period, including supplier names and payment terms.

SUPPLIER	PAYMENT_TERM	TOTAL_PAYMENT
Sahha Pharma Distributors	Monthly	450
Cedars Medical Supply Co.	Weekly	700
LebHealth Pharmaceuticals	Yearly	412.5
Lebanese Life Sciences Supply	Weekly	1500
AlphaMed Lebanon	Yearly	800
Phoenicia Medica	Weekly	540
MedCare Solutions Lebanon	Yearly	360
Mount Lebanon Drug Distributors	Monthly	480
Cedar Valley Pharma	Monthly	675
Beirut Pharma Solutions	Monthly	175

QUERY 10: High-Risk Customer Detection

Problem:

The Healer Dealer Pharmacy values customer care and high-quality service. However, we predict we will face a certain problem when dealing with controlled medication. We deeply care about our customers, and so when some have frequent purchases of controlled or prescription-only medications within a short period, this causes us great concern.

These high-frequency purchases of controlled medication may be indicative of addiction, drug abuse or misuse, or getting prescriptions from many different doctors.

However, keeping track of this manually is quite taxing and, frankly, difficult to do. So we decided to include a query that identifies high-risk customers who have brought at least more than 3 prescriptions in the last 30 days. It will also output their full name, phone number, number of prescriptions, and last purchase date. This information will help our employees decide when to intervene.

But first, as we don't have any high-risk customers in our database, we will insert some new values in MEDICINE, PRESCRIPTION, BILLING_INVOICE, and Contains2 to create some high-risk customers, allowing us to test this query properly:

- 1- We insert new MEDICINE tuples having Prescription_required = 'T' (true):

```
INSERT INTO MEDICINE VALUES ('J01JJ01', 'Oxycodone', 'Pain reliever', 'T', 20.00, 'PharmaLeb');
```

```
INSERT INTO MEDICINE VALUES ('K02KK01', 'Hydromorphone', 'Pain reliever', 'T', 25.00, 'Medico');
```

```
INSERT INTO MEDICINE VALUES ('L03LL01', 'Fentanyl', 'Pain reliever', 'T', 30.00, 'HealthPlus');
```

```
INSERT INTO MEDICINE VALUES ('M04MM01', 'Methadone', 'Opioid analgesic', 'T', 22.00, 'PharmaLeb');
```

```
INSERT INTO MEDICINE VALUES ('N05NN01', 'Lorazepam', 'Benzodiazepine', 'T', 18.00, 'Medico');
```

- 2- We also insert into our database various PRESCRIPTION values issued over the last 30 days for 3 customers: Sami Haddad, Rita Khalil, and Fadi Kassem. Due to the high number of prescriptions over a short time, these customers should appear as high risk in the final output.

-- Sami Haddad

```
INSERT INTO PRESCRIPTION VALUES (11, TO_DATE('05/11/2024', 'DD/MM/YYYY'), 1, TO_DATE('05/12/2024', 'DD/MM/YYYY'), '123456789', 'Dr. Fadi Kassem', 'DR12345', 30, 'Sami', 'Haddad');
```

```
INSERT INTO PRESCRIPTION VALUES (12, TO_DATE('10/11/2024', 'DD/MM/YYYY'), 1,
TO_DATE('10/12/2024', 'DD/MM/YYYY'), '123456789', 'Dr. Fadi Kassem', 'DR12345', 30, 'Sami',
'Haddad');
```

```
INSERT INTO PRESCRIPTION VALUES (13, TO_DATE('15/11/2024', 'DD/MM/YYYY'), 1,
TO_DATE('15/12/2024', 'DD/MM/YYYY'), '123456789', 'Dr. Fadi Kassem', 'DR12345', 30, 'Sami',
'Haddad');
```

```
INSERT INTO PRESCRIPTION VALUES (14, TO_DATE('20/11/2024', 'DD/MM/YYYY'), 1,
TO_DATE('20/12/2024', 'DD/MM/YYYY'), '123456789', 'Dr. Fadi Kassem', 'DR12345', 30, 'Sami',
'Haddad');
```

-- Rita Khalil

```
INSERT INTO PRESCRIPTION VALUES (15, TO_DATE('12/11/2024', 'DD/MM/YYYY'), 1,
TO_DATE('12/12/2024', 'DD/MM/YYYY'), '987654321', 'Dr. Hassan Amin', 'DR98765', 15, 'Rita',
'Khalil');
```

```
INSERT INTO PRESCRIPTION VALUES (16, TO_DATE('17/11/2024', 'DD/MM/YYYY'), 1,
TO_DATE('17/12/2024', 'DD/MM/YYYY'), '987654321', 'Dr. Hassan Amin', 'DR98765', 15, 'Rita',
'Khalil');
```

```
INSERT INTO PRESCRIPTION VALUES (17, TO_DATE('20/11/2024', 'DD/MM/YYYY'), 1,
TO_DATE('20/12/2024', 'DD/MM/YYYY'), '987654321', 'Dr. Hassan Amin', 'DR98765', 15, 'Rita',
'Khalil');
```

-- Fadi Kassem

```
INSERT INTO PRESCRIPTION VALUES (18, TO_DATE('22/11/2024', 'DD/MM/YYYY'), 1,
TO_DATE('22/12/2024', 'DD/MM/YYYY'), '112233445', 'Dr. Layla Zein', 'DR23456', 30, 'Fadi',
'Kassem');
```

```
INSERT INTO PRESCRIPTION VALUES (19, TO_DATE('24/11/2024', 'DD/MM/YYYY'), 1,
TO_DATE('24/12/2024', 'DD/MM/YYYY'), '112233445', 'Dr. Layla Zein', 'DR23456', 30, 'Fadi',
'Kassem');
```

```
INSERT INTO PRESCRIPTION VALUES (20, TO_DATE('25/11/2024', 'DD/MM/YYYY'), 1,
TO_DATE('25/12/2024', 'DD/MM/YYYY'), '112233445', 'Dr. Layla Zein', 'DR23456', 30, 'Fadi',
'Kassem');
```

```
INSERT INTO PRESCRIPTION VALUES (21, TO_DATE('28/11/2024', 'DD/MM/YYYY'), 1,
TO_DATE('28/12/2024', 'DD/MM/YYYY'), '112233445', 'Dr. Layla Zein', 'DR23456', 30, 'Fadi',
'Kassem');
```

- 3- We also insert these customers billing invoices into the BILLING_INVOICE table of our database, which will include further details of their purchase:

-- Sami Haddad

```
INSERT INTO BILLING_INVOICE VALUES (11, '123456789', TO_DATE('05/11/2024',
'DD/MM/YYYY'), 20.00, 'paid', 'Sami', 'Haddad', 20.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (12, '123456789', TO_DATE('10/11/2024',
'DD/MM/YYYY'), 20.00, 'paid', 'Sami', 'Haddad', 20.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (13, '123456789', TO_DATE('15/11/2024',
'DD/MM/YYYY'), 20.00, 'paid', 'Sami', 'Haddad', 20.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (14, '123456789', TO_DATE('20/11/2024',
'DD/MM/YYYY'), 20.00, 'paid', 'Sami', 'Haddad', 20.00);
```

-- Rita Khalil

```
INSERT INTO BILLING_INVOICE VALUES (15, '987654321', TO_DATE('12/11/2024',
'DD/MM/YYYY'), 25.00, 'paid', 'Rita', 'Khalil', 25.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (16, '987654321', TO_DATE('17/11/2024',
'DD/MM/YYYY'), 25.00, 'paid', 'Rita', 'Khalil', 25.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (17, '987654321', TO_DATE('20/11/2024',
'DD/MM/YYYY'), 25.00, 'paid', 'Rita', 'Khalil', 25.00);
```

-- Fadi Kassem

```
INSERT INTO BILLING_INVOICE VALUES (18, '112233445', TO_DATE('22/11/2024',
'DD/MM/YYYY'), 30.00, 'paid', 'Fadi', 'Kassem', 30.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (19, '112233445', TO_DATE('24/11/2024',
'DD/MM/YYYY'), 30.00, 'paid', 'Fadi', 'Kassem', 30.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (20, '112233445', TO_DATE('25/11/2024',
'DD/MM/YYYY'), 30.00, 'paid', 'Fadi', 'Kassem', 30.00);
```

```
INSERT INTO BILLING_INVOICE VALUES (21, '112233445', TO_DATE('28/11/2024',
'DD/MM/YYYY'), 30.00, 'paid', 'Fadi', 'Kassem', 30.00);
```

- 4- Finally, we must link the prescriptions to the medications, indicating what medicine the customers were prescribed:

-- Sami Haddad

```
INSERT INTO contains2 VALUES (11, 'J01JJ01');
```

```
INSERT INTO contains2 VALUES (12, 'J01JJ01');
```

```
INSERT INTO contains2 VALUES (13, 'K02KK01');
```

```
INSERT INTO contains2 VALUES (14, 'K02KK01');
```

-- Rita Khalil

```
INSERT INTO contains2 VALUES (15, 'K02KK01');
```

```
INSERT INTO contains2 VALUES (16, 'L03LL01');
```

```
INSERT INTO contains2 VALUES (17, 'N05NN01');
```

```
-- Fadi Kassem
```

```
INSERT INTO contains2 VALUES (18, 'N05NN01');
```

```
INSERT INTO contains2 VALUES (19, 'K02KK01');
```

```
INSERT INTO contains2 VALUES (20, 'L03LL01');
```

```
INSERT INTO contains2 VALUES (21, 'N05NN01');
```

Now we have created high risk customers! We can now start with our query that will detect them.

Query:

```
SELECT
```

```
    C.Fname,
```

```
    C.Lname,
```

```
    C.Phone_number,
```

```
    COUNT(P.PrescriptionID) AS Prescription_Count,
```

```
    MAX(B.Date_issued) AS Last_Purchase_Date
```

```
FROM
```

```
    CUSTOMER C,
```

```
    PRESCRIPTION P,
```

```
    BILLING_INVOICE B,
```

```
    MEDICINE M,
```

```
    contains2 CT
```

```
WHERE
```

```

C.Fname = P.CF_name
AND C.Lname = P.CL_name
AND P.PrescriptionID = CT.pID
AND CT.mID = M.MedicineID
AND M.Prescription_required = 'T'
AND B.InvoiceID = C.Bill_ID
AND B.Date_issued > (CURRENT_DATE - INTERVAL '30' DAY)

GROUP BY

C.Fname, C.Lname, C.Phone_number

HAVING

COUNT(P.PrescriptionID) > 3

ORDER BY

Last_Purchase_Date DESC;

```

Output: A list of high-risk customers, including their contact details, number of prescriptions, and the last purchase date, allowing pharmacists to investigate or intervene as needed.

FNAME	LNAME	PHONE_NUMBER	PRESCRIPTION_COUNT	LAST_PURCHASE_DATE
Fadi	Kassem	+96166666666	5	11/23/2024
Rita	Khalil	+96122222222	4	11/16/2024
Sami	Haddad	+96111111111	6	11/02/2024

Normalization Up to The BCNF Normal Form:

After creating all relations, we should improve them by normalizing according to several normal forms. Here we are going to normalize our database up to the fourth normal form, which is the Boyce-Codd

Normal Form (BCNF). On each relation, we are going to apply the four normal forms. We start with the first, then second, then third, and at last the BCNF normal form. Let us first start by a general description of each normal form.

First Normal Form (1NF):

This form does not allow multivalued attributes, composite attributes, or their combinations to exist in a relation.

1. Only attribute values permitted are single atomic values.
2. The domain of an attribute must only include atomic values, and the value of an attribute in a tuple must be a single value from the domain of that attribute.
3. It disallows having a set of values as an attribute value for a single tuple.

Second Normal Form (2NF):

The second normal form is based on the concept of **full functional dependency**. Before explaining the second form, let us define some concepts used in this form and others:

Functional Dependencies: A constraint between two sets of attributes in the database. The values of the Y component of a tuple in relation R depend on, or are determined by, the values of an X component. We say that Y is functionally dependent on X.

Prime Attribute: An attribute that is a member of a candidate key in relation R. An attribute is called **non-prime** if it is not a member of any candidate key.

Full Functional Dependency: A functional dependency $X \rightarrow Y$ is a full functional dependency if removing any attribute A from X means the dependency does not hold anymore.

Partial Dependency: A functional dependency $X \rightarrow Y$ is a partial functional dependency if removing any attribute A from X means the dependency still holds.

A relation schema R is in the second normal form if every non-prime attribute in R is fully functionally dependent on every key of R and every non-prime attribute A in R is not partially dependent on any key in R.

Third Normal Form (3NF):

The third normal form is based on the concept of **transitive dependency**. Let us first define transitive dependency:

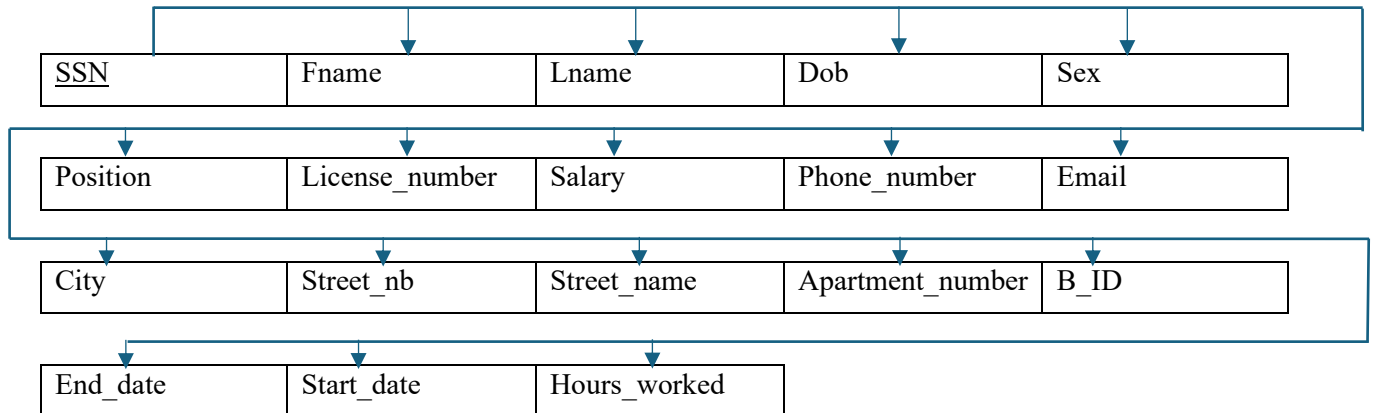
Transitive Dependency: A functional dependency $X \rightarrow Y$ in a relation schema R is a transitive dependency if there exists a set of attributes Z in R that is neither a candidate key nor a subset of any key of R , and both $X \rightarrow Z$ and $Z \rightarrow Y$ hold.

A relation schema R is in the third normal form if it satisfies the second normal form and no non-prime attribute of R is transitively dependent on the primary key. For every non-trivial functional dependency $X \rightarrow Y$, either X should be a superkey, or Y is a prime attribute.

Boyce-Codd Normal Form (BCNF):

The Boyce-Codd normal form is a stricter form than the third normal form. The BCNF differs from the definition of the third normal form in only one condition: The third normal form allows the right-hand side of the functional dependency to be a prime attribute, while BCNF does not allow that.

1. EMPLOYEE



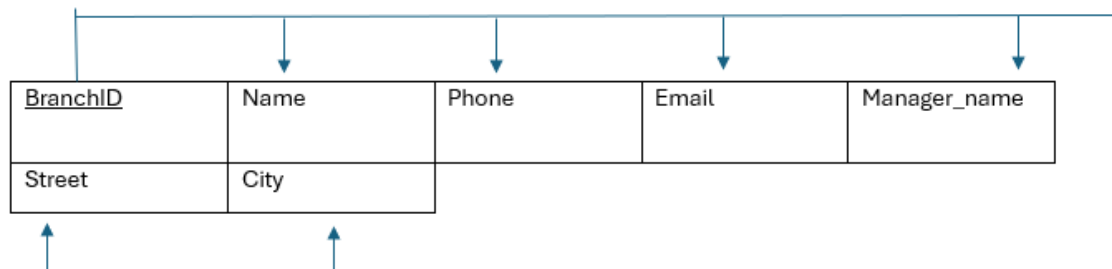
A. The **EMPLOYEE** relation schema satisfies all conditions of the 1NF because it has neither multivalued attributes nor composite attributes. All attributes are single and atomic.

B. The **EMPLOYEE** relation schema satisfies all conditions of the 2NF because every nonprime attribute is fully functionally dependent on the primary key “SSN”.

C. The **EMPLOYEE** relation schema satisfies all conditions of the 3NF because it satisfies the 2NF and there are no non-prime attributes that are transitively dependent on the primary key “SSN”.

D. The **EMPLOYEE** relation schema satisfies all conditions of the BCNF because there exists no functional dependency $X \rightarrow A$ where X is not a super key or A is a prime attribute and X is not a super key.

2. BRANCH



A. First Normal Form (1NF):

The Branch relation schema satisfies all conditions of the 1NF because it has neither multivalued attributes nor composite attributes. All attributes are single and atomic.

B. Second Normal Form (2NF):

The Branch relation schema satisfies all conditions of the 2NF because every nonprime attribute (Name, Phone, Email, Manager_name, Street, City) is fully functionally dependent on the primary key BranchID. There are no partial dependencies.

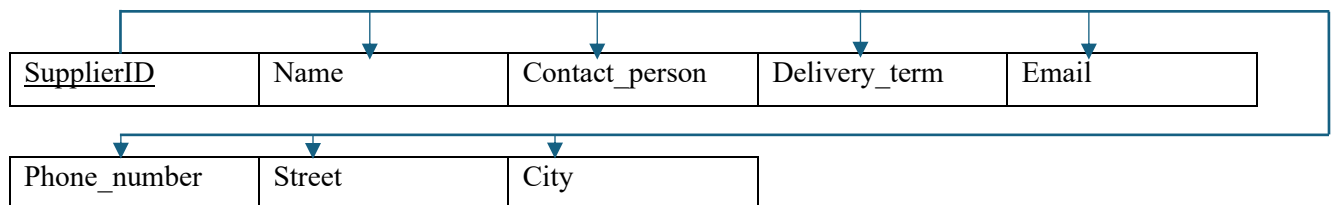
C. Third Normal Form (3NF):

The Branch relation schema satisfies all conditions of the 3NF because it satisfies 2NF and there are no non-prime attributes that are transitively dependent on the primary key BranchID.

D. Boyce-Codd Normal Form (BCNF):

The Branch relation schema satisfies all conditions of the BCNF because there exists no functional dependency $X \rightarrow A$ where X is not a super key or A is a prime attribute.

3. SUPPLIER



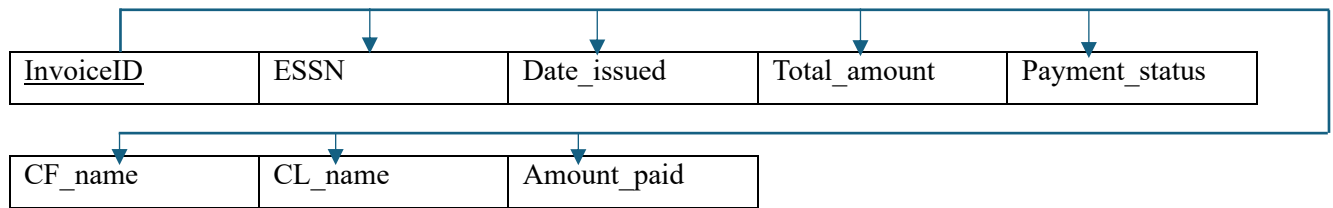
A. The **SUPPLIER** relation schema satisfies all conditions of the 1NF because it has neither multivalued attributes nor composite attributes. All attributes are single and atomic.

B. The **SUPPLIER** relation schema satisfies all conditions of the 2NF because every nonprime attribute is fully functionally dependent on the primary key “SupplierID”.

C. The **SUPPLIER** relation schema satisfies all conditions of the 3NF because it satisfies the 2NF and there are no non-prime attributes that are transitively dependent on the primary key “SupplierID”.

D. The **SUPPLIER** relation schema satisfies all conditions of the BCNF because there exists no functional dependency $X \rightarrow A$ where X is not a super key or A is a prime attribute and X is not a super key.

4. BILLING_INVOICE



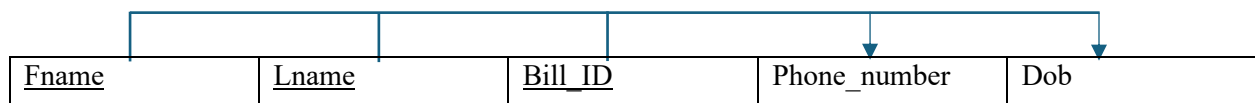
A. The **BILLING_INVOICE** relation schema satisfies all conditions of the 1NF because it has neither multivalued attributes nor composite attributes. All attributes are single and atomic.

B. The **BILLING_INVOICE** relation schema satisfies all conditions of the 2NF because every nonprime attribute is fully functionally dependent on the primary key “InvoiceID”.

C. The **BILLING_INVOICE** relation schema satisfies all conditions of the 3NF because it satisfies the 2NF and there are no non-prime attributes that are transitively dependent on the primary key “InvoiceID”.

D. The **BILLING_INVOICE** relation schema satisfies all conditions of the BCNF because there exists no functional dependency $X \rightarrow A$ where X is not a super key or A is a prime attribute and X is not a super key.

5. CUSTOMER



A. First Normal Form (1NF):

The Customer relation schema satisfies all conditions of 1NF because it has neither multivalued attributes nor composite attributes. All attributes are single and atomic.

B. Second Normal Form (2NF):

The Customer relation schema satisfies all conditions of 2NF because every non-prime attribute (Phone_number, Dob) is fully functionally dependent on the composite primary key (Fname, Lname, Bill_ID).

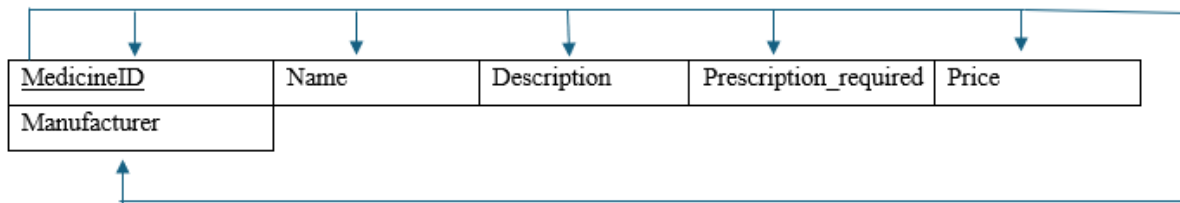
C. Third Normal Form (3NF):

The Customer relation schema satisfies all conditions of 3NF because it satisfies 2NF and there are no non-prime attributes transitively dependent on the composite primary key (Fname, Lname, Bill_ID).

D. Boyce-Codd Normal Form (BCNF):

The Customer relation schema satisfies all conditions of BCNF because there exists no functional dependency $X \rightarrow A$ where X is not a superkey, or A is a prime attribute and X is not a superkey.

6. MEDICINE



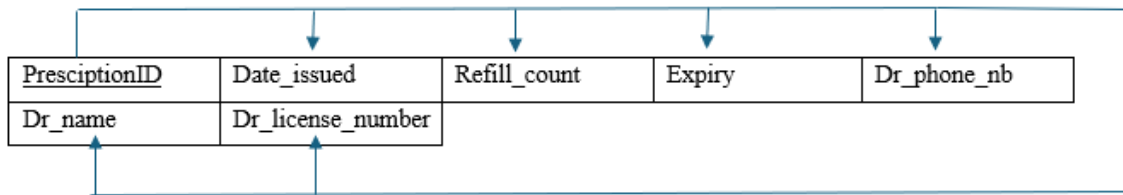
A. First Normal Form (1NF): The Medicine relation schema satisfies all conditions of the 1NF because it has neither multivalued attributes nor composite attributes. All attributes are single and atomic.

B. Second Normal Form (2NF): The Medicine relation schema satisfies all conditions of the 2NF because every nonprime attribute (Name, Description, Prescription_required, Price, Manufacturer) is fully functionally dependent on the primary key MedicineID. There are no partial dependencies.

C. Third Normal Form (3NF): The Medicine relation schema satisfies all conditions of the 3NF because it satisfies 2NF and there are no non-prime attributes that are transitively dependent on the primary key MedicineID.

D. Boyce-Codd Normal Form (BCNF): The Medicine relation schema satisfies all conditions of the BCNF because there exists no functional dependency $X \rightarrow A$ where X is not a super key or A is a prime attribute

7. PRESCRIPTION



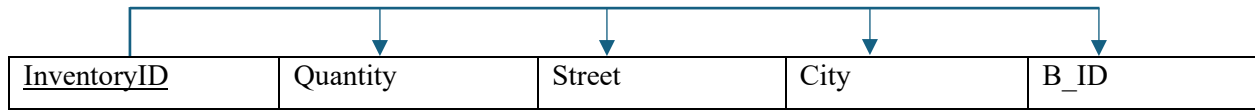
A. First Normal Form (1NF): The Prescription relation schema satisfies all conditions of the 1NF because it has neither multivalued attributes nor composite attributes. All attributes are single and atomic.

B. Second Normal Form (2NF): The Prescription relation schema satisfies all conditions of the 2NF because every nonprime attribute (DateIssued, Refill_Count, Expiry, Dr_Phone_nb, Dr_name, Dr_license_number) is fully functionally dependent on the primary key PrescriptionID. There are no partial dependencies.

C. Third Normal Form (3NF): The Prescription relation schema satisfies all conditions of the 3NF because it satisfies 2NF and there are no non-prime attributes that are transitively dependent on the primary key PrescriptionID.

D. Boyce-Codd Normal Form (BCNF): The Prescription relation schema satisfies all conditions of the BCNF because there exists no functional dependency $X \rightarrow A$ where X is not a super key or A is a prime attribute.

8. INVENTORY



A. First Normal Form (1NF):

The Inventory relation schema satisfies all conditions of 1NF because it has neither multivalued attributes nor composite attributes. All attributes are single and atomic.

B. Second Normal Form (2NF):

The Inventory relation schema satisfies all conditions of 2NF because every non-prime attribute (Quantity, Street, City, B_ID) is fully functionally dependent on the primary key InventoryID.

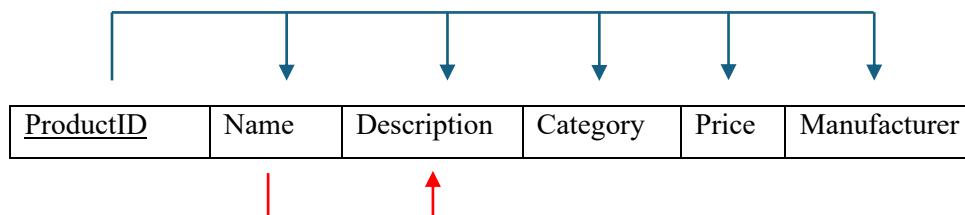
C. Third Normal Form (3NF):

The Inventory relation schema satisfies all conditions of 3NF because it satisfies 2NF and there are no non-prime attributes that are transitively dependent on the primary key InventoryID.

D. Boyce-Codd Normal Form (BCNF):

The Inventory relation schema satisfies all conditions of BCNF because there exists no functional dependency $X \rightarrow A$ where X is not a superkey, or A is a prime attribute and X is not a superkey.

9. SELF_CARE_HYGIENE_PRODUCT

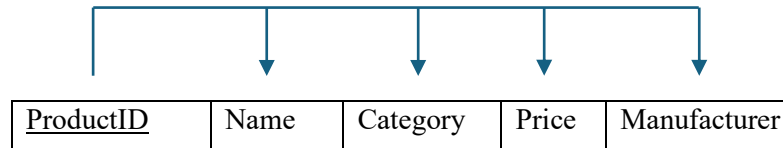


A. The **SELF_CARE_HYGIENE_PRODUCT** relation schema satisfies the first normal form “1NF” since its arguments are neither multivalued nor composite. All attributes are single and atomic.

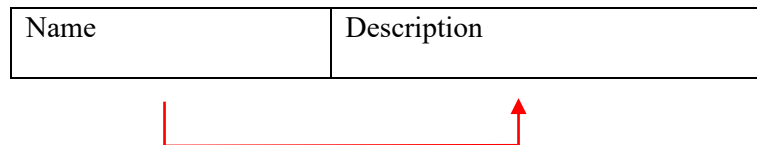
B. The **SELF_CARE_HYGIENE_PRODUCT** relation schema satisfies the second normal form “2NF” since all the nonprime attributes are fully functionally dependent on the primary key ProductID.

C. The **SELF_CARE_HYGIENE_PRODUCT** relation schema doesn't satisfy all the conditions in the Third Normal Form "3NF" since the functional dependency represented by **Name $\rightarrow$ Description** is a functional dependency where neither Name is a super key nor Description is a prime attribute. Thus, this needs further decomposition:

SELF_CARE_HYGIENE_PRODUCT_A:

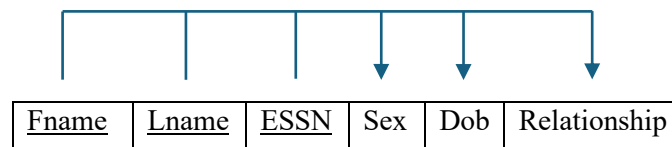


SELF_CARE_HYGIENE_PRODUCT_B:



D. The **SELF_CARE_HYGIENE_PRODUCT** relation schema satisfies all the conditions of the BCNF since there is no functional dependency $X \rightarrow A$ where X is neither a super key nor A is a prime attribute and X is not a super key.

10. DEPENDENT



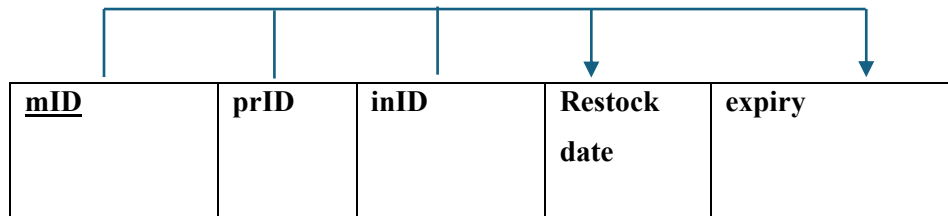
A. The **DEPENDENT** relation schema satisfies all conditions of the First Normal Form "1NF" since it has neither multivalued nor composite attributes. All attributes are single and atomic.

B. The **DEPENDENT** relation schema satisfies all conditions of the Second Normal Form "2NF" since every nonprime attribute is fully functionally dependent on the primary key Fname, Lname, and ESSN.

C. The **DEPENDENT** relation schema satisfies all conditions of the Third Normal Form "3NF" since it first satisfies the "2NF", and secondly there are no nonprime attributes that are transitively dependent on the primary key Fname, Lname, and ESSN.

D. The **DEPENDENT** relation schema satisfies all conditions of BCNF since there is no functional dependency $X \rightarrow A$ where X is not a super key or A is a prime attribute and X not a super key.

11. STORES



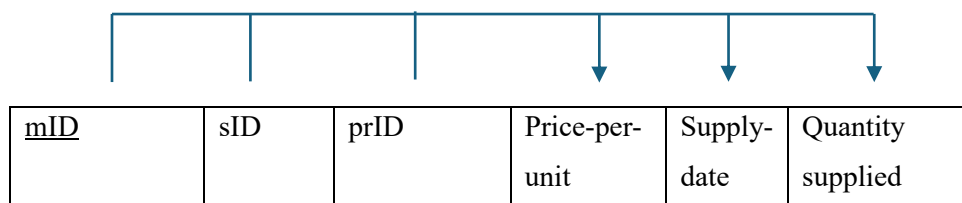
A. The **STORES** relation schema satisfies the **First Normal Form (1NF)** because all attributes are single and atomic, with no multivalued or composite attributes.

B. The **STORES** relation schema satisfies the **Second Normal Form (2NF)** because all nonprime attributes (Restock_date, Expiry) are fully functionally dependent on the composite primary key (mID, prID, inID).

C. The **STORES** relation schema satisfies the **Third Normal Form (3NF)** because it satisfies 2NF, and no transitive dependencies exist among its attributes.

D. The **STORES** relation schema satisfies **BCNF** because there is no functional dependency $X \rightarrow A$ where X is not a super key or A is a prime attribute and X not a super key.

12. SUPPLIES



A. The **SUPPLIES** relation schema satisfies the **First Normal Form (1NF)** because all attributes are single and atomic, with no multivalued or composite attributes.

B. The **SUPPLIES** relation schema satisfies the **Second Normal Form (2NF)** because all nonprime attributes (Price_per_unit, Supply_date, Quantity_supplied) are fully functionally dependent on the composite primary key (mID, sID, prID).

C. The **SUPPLIES** relation schema satisfies the **Third Normal Form (3NF)** because it satisfies 2NF, and no transitive dependencies exist among its attributes.

D. The **SUPPLIES** relation schema satisfies **BCNF** because there is no functional dependency $X \rightarrow AX \setminus A$ where AX is not a super key, ensuring full compliance with BCNF.

Conclusion

In this report, we aimed to meticulously design and implement a robust and efficient database for the Healer Dealer Pharmacy. The development of this database required breaking the process into distinct phases due to the intricate and sensitive nature of database design.

In Phase I, we focused on identifying and designing the key entities and relationships that form the foundation of the database. Moving to Phase II, we translated these high-level descriptions into relational schemas that accurately represent the structure of the data. In Phase III, these schemas were transformed into SQL code, allowing us to implement the queries and create the necessary tables in the Oracle server. Finally, in Phase IV, we normalized the database to ensure efficiency and reduce redundancy, applying the Boyce-Codd Normal Form (BCNF).

A well-designed database is essential for any organization, regardless of its size. It must be carefully constructed to ensure that it accurately reflects the mini-world of the business it represents, in this case, a pharmacy. By handling the various relationships and transactions with precision, we ensure the reliability, integrity, and functionality of the database for future users. This guarantees that the Healer Dealer Pharmacy's data will be represented accurately and efficiently, ready to support the day-to-day operations of the business.